FlakyFix: Using Large Language Models for Predicting Flaky Test Fix Categories and Test Code Repair

Sakina Fatima, Hadi Hemmati, and Lionel C. Briand, *Fellow, IEEE*

***Abstract*—Flaky tests are problematic because they non-deterministically pass or fail for the same software version under test, causing confusion and wasting development effort. While machine learning models have been used to predict flakiness and its root causes, there is much less work on providing support to fix the problem. To address this gap, in this paper, we focus on predicting the type of fix that is required to remove flakiness and then repair the test code on that basis. We do this for a subset of flaky tests where the root cause of flakiness is in the test itself and not in the production code. One key idea is to guide the repair process with additional knowledge about the test's flakiness in the form of its predicted fix category. Thus, we first propose a framework that automatically generates labeled datasets for 13 fix categories and trains models to predict the fix category of a flaky test by analyzing the test code only. Our experimental results using code models and few-shot learning show that we can correctly predict most of the fix categories. To show the usefulness of such fix category labels for automatically repairing flakiness, we augment the prompts of GPT 3.5 Turbo, a Large Language Model (LLM), with such extra knowledge to request repair suggestions. The results show that our suggested fix category labels, complemented with in-context learning, significantly enhance the capability of GPT 3.5 Turbo in generating fixes for flaky tests. Based on the execution and analysis of a sample of GPT-repaired flaky tests, we estimate that a large percentage of such repairs, (roughly between 51% and 83%) can be expected to pass. For the failing repaired tests, on average, 16% of the test code needs to be further changed for them to pass.**

***Index Terms*—Flaky tests, fix category, test repair, large language models, code models, few shot learning, software testing.**

Received 13 January 2024; revised 30 August 2024; accepted 27 September 2024. Date of publication 2 October 2024; date of current version 12 December 2024. This work was supported in part by a Research Grant from Huawei Technologies Canada, Mitacs Canada and in part by Canada Research Chair and Discovery Grant programs of the Natural Sciences and Engineering Research Council of Canada (NSERC). The work of Hadi Hemmati was supported by the NSERC and Alberta Innovates. The work of Lionel C. Briand was supported by the Science Foundation Ireland under Grant 13/RC/2094-2. Recommended for acceptance by Y. Xiong. *(Corresponding author: Sakina Fatima.)*

Sakina Fatima is with the School of EECS, University of Ottawa, Ottawa, ON K1H 8M5, Canada (e-mail: sfati077@uottawa.ca).

Hadi Hemmati is with the Electrical Engineering and Computer Science Department, York University, Toronto, ON M3J 1P3, Canada (e-mail: hemmati@yorku.ca).

Lionel C. Briand is with the School of EECS, University of Ottawa, Canada, and also with the Lero SFI Centre for Software Research, University of Limerick, V94 T9PX Limerick, Ireland (e-mail: lbriand@uottawa.ca).

Digital Object Identifier 10.1109/TSE.2024.3472476

I. Introduction

FLAKY tests are test cases that non-deterministically fail or pass in different runs, without any changes in the source code being tested. This inconsistency is frustrating and troublesome for developers and Quality Assurance (QA) teams, especially during regression testing. Oftentimes, flakiness results from multiple factors, including variations in configurations used to execute identical code, timing issues influenced by prior executions, race conditions in concurrent systems, and absent external dependencies.

Flaky tests are a widespread problem in software development and a major challenge to address. With millions of tests in repositories, even a small percentage of flaky tests require significant manual effort to detect and fix. For example, a report from Google shows that they had 1.6M test failures on average each day, and 73K of them (4.56%) were caused by flaky tests [1], [2]. Several other companies like Microsoft [3], Facebook [4], Netflix, Mozilla, and Salesforce [5] have reported similar experiences, where flaky tests have significantly delayed their deployment process.

To detect test flakiness, according to a recent study [6], practitioners are still relying mainly on reruns and manual inspection. In the literature, state-of-the-art (SOTA) work on handling test flakiness relies on both static techniques [7], [8], [9], [10] and advanced machine learning (ML) models [11], [12], [13], [14], [15], [16], [17], [18], [19] for the automated prediction of flakiness and its causes. However, there is limited work [20] on helping developers and testers fix the flakiness of their test cases.

Flakiness in tests can stem from highly diverse reasons [21] and can be resolved through various strategies, as evident from the International Dataset of Flaky Tests (IDoFT) [22], the largest open-source repository containing Java and Python flaky tests and their respective fixes. Remedies range from altering the test execution order to addressing operating system or implementation dependencies by code modifications in the test class. Our study focuses on repairing flaky tests in a black-box manner (no access to production code) by updating flaky statements within the test case code, which constitutes 10%

© 2024 The Authors. This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

(562 tests from accepted pull requests) of the total 5,500 tests in the Java dataset [22]. While such flaky tests represent a minority in this dataset, techniques to fix them are warranted, thereby serving the important needs of the testing community in charge of writing and evolving the test code. In addition, studies such as that of Akli et al. [23] report that, in their dataset, 70% of flakiness issues originate in the test code [1], [24], further supporting our focus. The distinction between fixing the production code and the test code is crucial. Considering the production code would dramatically increase the costs and complexity of repair, as it would warrant code coverage data collection during test execution for fault localization purposes. This is particularly challenging for complex production code with numerous dependencies and often multiple programming languages [12].

More precisely, our goal is to (a) define different categories of fixes that are applied to the test code to remove flakiness; (b) given a flaky test case (and without the production code), predict the most suitable fix category to apply; and (c) incorporate this predicted fix category into an automated flaky test repair approach.

Our study is different than existing work [23] whose focus is on predicting the category of flaky tests with respect to their root causes. Since knowing only root causes, especially at a high-level, does not necessarily help in fixing the test code, our work focuses on predicting fix categories in terms of what part of the test code needs fixing. Therefore, such fix categories are expected to guide the tester towards concrete modifications and thus decrease their manual investigation effort. Note that these categories can be useful for both human developers and testers to manually fix flakiness and for tools to automatically and fully repair flaky test cases. In this paper, however, we only report on the latter.

To achieve the above three goals, our approach consists of three steps: (a) Qualitatively analysing flaky test datasets and proposing a list of fix categories to remove test flakiness, when its root cause is in the test code only; (b) Conducting empirical analyses to explore the use of pre-trained code models in predicting the necessary fix category for addressing identified flaky tests; and (c) Evaluating the improvements such categories can provide to an LLM such as GPT when asked to generate fixes for flaky tests.

In step (b) we assess two smaller code models, CodeBERT [25], and UniXcoder [26], which are specifically tailored for smaller datasets and are pre-trained with a reduced parameter set (125 million). We fine-tuned these models with and without Few-Shot Learning (FSL) [27]. Considering FSL stems from commonly dealing with small datasets of flaky tests within each fix category, which are typically available for the fine-tuning process.

In step (c), we evaluate the performance of recent Large Language Models (LLMs), namely GPT [28], which possess a vast number of model parameters (up to 175 billion in GPT 3.5 Turbo), in analyzing their capability to generate fixes for flaky tests. The steps (b) and (c) are depicted in Fig. 1. Our findings indicate several key points: (1) Smaller pre-trained code models demonstrate a high accuracy in predicting most fix categories, achieving precision rates ranging from 83% to 98% and recall rates from 67% to 97%, solely by analyzing the test code; (2) UniXcoder surpasses CodeBERT in predicting fix categories; (3) The inclusion of FSL does not yield significant improvements in the results; (4) Our predicted fix category labels play a significant role in guiding GPT to generate better full fixes for flaky tests. We evaluated this using CodeBLEU [29] and achieved a mean score of 81% when using predicted fix category labels in the input prompt, compared to 75% without them. These results are further enhanced with in-context learning [30], which we tested on three different fix categories; (5) Based on the execution and analysis of a sample of GPT-repaired tests, we estimate that a large percentage can be expected to pass, roughly between 46% and 80% of them. For the generated tests that fail, the required changes to correct them are limited, with an average of 16% of the tokens needing modification in the test code. In summary, the contributions of this paper are as follows:

- Definition of a categorization for flaky test fixes. The goal is to provide practical guidance for fixing tests.
- Development of a set of heuristics (search rules) and accompanying open-source scripts to automatically label flaky tests based on their fixes.
- Creation of a public dataset of labeled flaky tests for training, using our automated script.
- Definition of a black-box framework for predicting flaky test fix categories using code models (language models that are pre-trained on code) and few-shot learning.
- Quantitative and qualitative analysis of two SOTA small code models, with and without FSL, to compare alternative prediction models.
- Automatic generation of fixes for flaky tests using a LLM (GPT 3.5 Turbo), with and without the predicted fix categories, to analyze how these labels enhance the generated fixes.
- Based on the execution of a sample of GPT-generated tests, we rely on CodeBLEU metrics and statistical methods, including bootstrapping and logistic regression, to estimate the pass rate for all generated repaired tests, including those that could not be executed. We also assess the magnitude of the modifications needed to manually fix tests that failed.

The rest of this paper is organized as follows: Section II presents our approach for developing an automated tool for labeling flaky tests with fix categories and predicting such categories using code models, with and without FSL. It also describes how we incorporate such labels as part of prompts for GPT to create a fully automated flaky test repair mechanism. Section III evaluates our approach, reports experimental results, and discusses their implications. Section IV reports on a qualitative analysis of the two small code models and the LLM to demonstrate and explain their ability to learn from test code and subsequently make predictions and generate fixes. Section V analyzes threats to the validity of this study. Section VI reviews

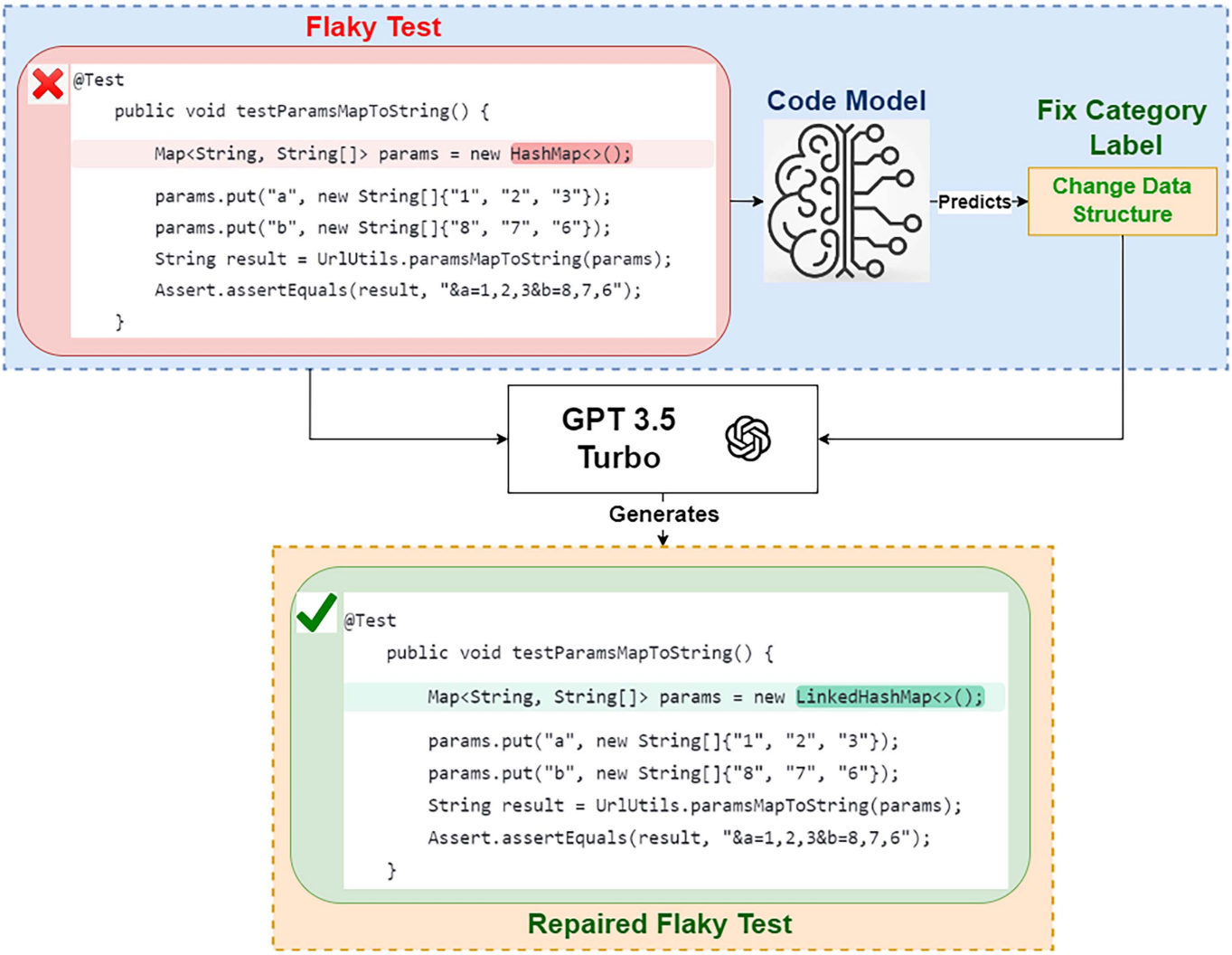


Fig. 1. Predicting fix category label and generating repaired flaky tests using large language models.

and contrasts related work. Finally, Section VII concludes the paper and suggests future work.

II. Approach

The approach of this paper is divided into three phases: (a) automatically constructing a high-quality dataset of fix categories, based on existing datasets of flaky test cases and their fixes; (b) building a prediction model of fix categories, using the created dataset, to guide the repair of flaky tests; and (c) automatically generating the fixed version of a given flaky test using LLMs, with the help of our predicted fix categories.

A. Automatically Constructing a Dataset of Fix Categories

Having an appropriate and well-defined methodology for constructing a high-quality dataset of fix categories is important in practice, as we expect that local prediction models will need to be built in most contexts based on locally collected data.

In a nutshell, our approach starts with an existing set of fix categories and then extends it to a more comprehensive and practical set, and finally devise heuristics to automatically label fixes in existing flaky test datasets.

To the best of our knowledge, the categories of flaky test fixes presented by Parry et al. [20], as shown in Table I, represent the largest and most recent classification in the literature. These fix categories were manually assessed by the authors by analyzing code diffs, developer's comments, and any linked issues of 75 flakiness-repairing commits that originated from 31 open-source Python projects [20].

Their categorization procedure accepts multiple sources of information (i.e., fixes but also comments and linked issues) to link a flaky test with a fix category. This information might not be entirely available for many flaky tests in a project. Therefore, following that procedure, not all fixes can be labeled with a category if we have only access to the flaky tests and their fixes, which is the case for many datasets.

In this paper, we aim to categorize fixes and train a prediction model whose objective is to assign a fix category to a new flaky test, without any other information than the test case's code. In other words, the goal is to guide testers (or automated repair

TABLE I
EACH FLAKY TEST IS ASSIGNED A CATEGORY DESCRIBING THE CAUSE OF FLAKINESS (ROWS) AND ANOTHER DESCRIBING THE REPAIR APPLIED BY DEVELOPERS (COLUMNS). FROM PARRY ET AL. [20]

Cause	Add Mock	Add/Adjust Wait	Guarantee Order	Isolate State	Manage Resource	Reduce Random	Reduce Scope	Widen Assertions	Miscellaneous	Total
Async. Wait	1	6	-	-	-	-	-	2	-	9
Concurrency	-	2	-	-	2	-	-	2	2	8
Floating Point	-	-	-	-	-	-	-	3	-	3
I/O	-	-	-	-	-	-	-	-	-	-
Network	3	3	-	-	1	-	-	-	1	8
Order Dependency	-	-	-	2	-	-	1	-	-	3
Randomness	-	-	-	-	-	6	-	4	1	11
Resource Leak	-	-	-	-	2	-	1	1	-	4
Time	5	-	-	-	-	-	1	1	2	9
Unordered Coll.	-	-	3	-	-	-	-	-	-	3
Miscellaneous	2	-	1	-	1	-	-	6	7	17
Total	11	11	4	2	6	6	3	19	13	75

tools) in applying the right fix according to categories of fixes that are predictable based on the code of the test cases alone. While not all flaky tests can be resolved solely by addressing the test code, a subset can indeed be fixed in this manner. Automating these fixes can still save time and resources. Hence, our dataset creation is targeting these specific fixes. To achieve that, we analyzed flaky test cases and their fixes, to extract ground truth labels. Inspired by Parry et al.'s work [20], we analyzed the International Dataset of Flaky Tests (IDoFT)[1], which comprises flaky test cases along with their fixes from 123 open-source Java projects.

This dataset, utilized in various prior studies on flaky test case prediction [5], [12], [31], [32], [33], [34], [35], is a publicly accessible dataset regularly receiving updates. We ensured our usage involved the most recent version of this dataset.

To assign labels to IDoFT dataset with fix categories, two researchers manually and independently analyzed 100 randomly selected flaky tests and their corresponding fixes to map them to one or more categories from those proposed by Parry et al. (see Table I) or a new category when the fix did not match any of them. The new categories should be predictable by only looking at the flaky tests and yet be useful enough to help developers and testers fix them. Differences in categories and definitions between the researchers were resolved through multiple meetings. We use Cohen's Kappa [36] to measure the initial level of agreement between the two researchers. This is briefly discussed in Section III. There was no case where a third opinion was required to resolve a conflict.

Some of the fix categories proposed by Parry et al. are also renamed in Table I for better understandability. For example, the term *Reduce Scope* (one of the Parry et al.'s categories) lacks clarity in indicating the exact code section that requires modification. By renaming *Reduce Scope* to *Change Condition*, developers and testers are more explicitly instructed to modify conditional statements. The process resulted in 13 different fix categories for flaky tests, as shown in Table II.

[1]https://mir.cs.illinois.edu/flakytests

TABLE II
COUNT OF FLAKY TESTS WITHIN EACH FIX CATEGORY, USING AUTOMATED LABELING. CATEGORIES WITH OVER 30 INSTANCES ARE HIGHLIGHTED

Flaky Test Fix Category	Number of Flaky Tests
Change Assertion	197
Change Condition	121
Reset Variable	104
Reorder Data	92
Change Data Structure	91
Handle Exception	52
Miscellaneous	50
Change Data Format	46
Reorder Parameters	37
Call Static Method	9
String Matching	6
Change Timezone	3
Handle Timeout	2

The final step in this phase was to create a set of heuristics to automatically label each fix from our dataset to one of these 13 categories. Each heuristic is a search rule (implemented as a Python script) that relies on keywords from the *diff* of a flaky test case and its fixed version. Subsequently, we manually verified the outputs of the automated labeling process to ensure the accuracy of the heuristics for future datasets. The complete labeling process is shown in Fig. 2. Additionally, after this final step, the remaining 462 tests from our dataset were automatically labeled using the developed heuristics. Note that a single test may be labeled with multiple fix categories, as illustrated in Table II.

In the following, we briefly explain each category's heuristic along with few examples from the IDoFT dataset.

For the **Change Assertion** category, the heuristic looks for the assert keyword and its replacements within the fixed flaky test code. For instance, if *AssertEquals*, *AssertThat*, or *AssertTrue* appear in deleted lines and *AssertJsonStringEquals*, *assertJSONEqual*, *AssertJsonEqualsNonStrict*, or *JsonAssert.AssertEquals* are present in the added lines, it shows an assertion change to address flakiness.

A common instance of flakiness arises due to the use of an incorrect assertion type. For example, *assertJSONEqual* should be used instead of *AssertThat* in an assert function that retrieves a JSON object from other functions in the test code and compares it with a hard-coded JSON String to ensure the deterministic ordering of the retrieved JSON object. An example illustrating the appropriate assertion type usage can be found online[2].

Changing data structures can be an effective technique to improve test reliability and reduce flakiness [37], [38]. For example, the data structures *Hash Map*, *Set*, *HashMap*, and

[2]https://github.com/vipshop/vjtools/pull/168

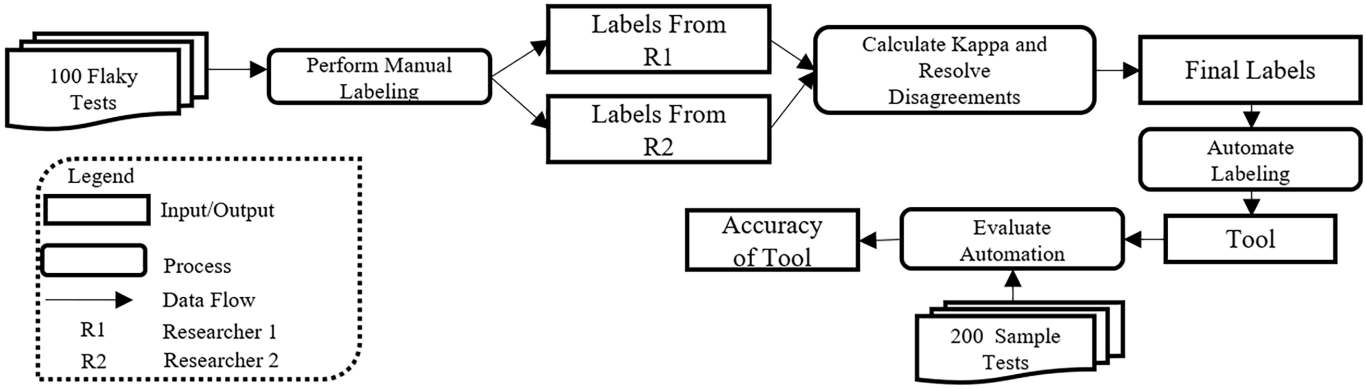


Fig. 2. Automated labeling of flaky tests with their respective fix categories.

HashSet are unordered collections. Their use in the test code can lead to test flakiness if the test relies on the order of elements in the structure. In such cases, these data structures should be replaced with *LinkedHash*, *LinkedMap*, *LinkedSet*, *LinkedHashMap* and *LinkedHashSet*, respectively, as they maintain the order in which their elements are stored, regardless of how many times a test is executed. This makes the test more reliable and less prone to flakiness. If *Hash*, *Map*, *Set*, *HashMap*, and *HashSet* keywords are replaced in the test code 'diff' with *LinkedHash*, *LinkedMap*, *LinkedSet*, *LinkedHashMap* and *LinkedHashSet*, respectively, then such test cases are labeled as **Change Data Structure**. An example of this type of fix can be found online[3].

Exception handling is usually an important tool to reduce unwanted behavior due to wrong inputs or scenarios and thus decrease flakiness. If there are *Try Catch* statements added or removed or changes occur in the type of the Exception used in the test case, the test case is labeled as **Handle Exception**. Handling exceptions, similar to other fix categories, can be combined with different fix strategies to entirely eliminate the root cause of flakiness within a test case. Examples of this can be seen online[4,5] where flakiness is fixed in the test codes after integrating exception handling techniques alongside alterations in assertion types.

A flaky test is labeled with the **Reset Variable** category if one of the keywords *reset*, *clear*, *remove* or *purge* is found in the *diff*. These are useful commands to reset the state of the system and clear the memory at the end of the test cases. They put the system back to its initial testing state, which is required for many tests to avoid flakiness. An example of this type of fix can be found online[6].

Input formatting mismatches can be a cause of flakiness. Oftentimes, a fix is simply changing the input variable's format to the required one. If there is change in the format of the string or numeric data that is used in a function of the test code, the test case is labeled as **Change Data Format**.

If conditional statements are added or removed in the test code, for example replacing the *containsexactly* keyword with the *containsexactlyinanyorder* keyword, the test case is labeled with the **Change Condition** category. An example of this fix is shown in this pull request[7]. If there is any of the keywords *sortfield*, *sort_properties*, *sorted*, *order by* or *sort* in the fixed test code, which is used to sort or change the order of data, then the test is labeled as **Reorder Data**. An example of this type of fix can be found online[8].

If there is a change in the order of the function parameters, then the **Reorder Parameters** label is used. An example of this type of fix can be found online[9].

Tests may rely on specific dates or times, which can be affected by time zone differences, leading to inconsistent test results. Setting a specific time zone ensures consistent results, increasing test reliability and reducing flakiness [1], [39]. If there is a change in the assigned value to the *timezone* keyword, then such tests are labeled as **Change Time Zone**.

If the *Timeout* keyword is added to the test code, which ensures that tests are completed within a reasonable time frame, thus reducing the likelihood of flakiness, then these test cases are labeled as **Handle Timeout**. This fix has been suggested in the work of Pei et al. [40] to fix flakiness related to Async Wait.

Static methods can help ensure consistent test behavior across multiple runs by encapsulating complex scenarios or behavior simulations, such as network failures or time delays. If there is a static method call added to the test code, then these tests are labeled as **Call Static Method**. If two strings are matched in a given test to make sure they are in the correct order for maintaining determinism by using keywords like *'match.that(s).isequalto* or *.matches* then these tests are marked as **String Matching**. An example of this type of fix can be found online[10]. Lastly, if there is some change in the added or deleted lines of code that do not belong to any of the above categories, the test cases are labeled as **Miscellaneous**. Examples

[3]https://github.com/biojava/biojava/pull/897
[4]https://github.com/apache/incubator/pull/398
[5]https://github.com/vipshop/vjtools/pull/168/files
[6]https://github.com/apache/nifi/pull/6754
[7]https://github.com/apache/servicecomb-java-chassis/pull/3393
[8]https://github.com/cerner/common-kafka/pull/50
[9]https://github.com/apache/nifi/pull/6709/files
[10]https://github.com/immutables/immutables/pull/1350

of such fixes that deviate from established patterns in other fix categories can be located online[11,12].

B. Flaky Test Case Fix Category Prediction With Code Models

In this section, we describe how we use different language models for predicting the fix category of a flaky test. This prediction may be not only useful to help testers identify the required fix but can also guide any automated repair process.

We use two different code models CodeBERT and UniXcoder as our prediction models, respectively. In general, language models are pre-trained using self-supervised learning, on a large corpus of unlabelled data. They are then further fine-tuned using a specific, labeled dataset for different Natural Language Processing (NLP) tasks such as text classification, relation extraction, or next-sentence prediction [25]. Code models are those language models that are specifically pre-trained on programming languages. CodeBERT and UniXCoder have been pre-trained using a diverse corpus encompassing code from six programming languages: Java, Python, JavaScript, PHP, Ruby, and Go. Both models excel in code classification tasks when compared to other pre-trained code models [12], [41]. These models comprehend both the syntax and semantics of various programming languages. We individually fine-tune these code models on our dataset of flaky tests to learn the underlying patterns of test code flakiness. During inference, these fine-tuned models classify flaky tests based on what these code models learned during training (fine-tuning). We choose not to fine-tune Large Language Models (LLMs) like GPT or CodeLlama for fix category prediction due to the high costs associated with their fine-tuning. In addition, as we will see in the results section the smaller code models will already provide very high accuracy which leaves no reason for us to switch to much more complex and heavier models. This process demands extensive data and substantial computational resources [42]. Reported studies show that smaller code models work well for classifying code [41] and are more cost-effective. Their efficient classification abilities, along with reduced time and computational expenses for refinement, make them more practical for integration into solutions for large-scale industrial systems.

For fine-tuning CodeBERT and UniXcoder to predict flaky test fix categories, we use two different techniques. In the first one, we augment the models with FSL using a Siamese Network to deal with the usually small datasets of flaky tests that are labeled in a typical development context. In contrast to FlakyCat, that also uses FSL [23], we train our model on the dataset created following the procedure presented in Section II-A, where each data item is a pair of flaky test and its fix category. Note that we do not use information from the actual fix during inference time, since our goal is to predict the fix category for a given flaky test which has not been repaired yet. Thus, in practice, only the test code is available to the model. In the second technique, we fine-tuned the models using a simple Feed Forward Neural Network (FNN) as done in earlier work for flaky test classification [12].

Furthermore, note that we do not build a multi-class prediction model to deal with multiple fix categories. Instead, we build multiple binary classification models to estimate the probability of whether a test belongs to each category. This is because flaky tests may belong to multiple fix categories. With multiple binary classifications, each fix category is treated separately, allowing the model to learn the features that are most relevant for that fix category. Since our dataset is relatively small, it is not computationally expensive to build these multiple classifiers for each category. However, as our dataset expands to include more samples, we may consider transitioning to a multi-class classifier for efficiency and scalability purposes in the future.

1) Converting Test Code to Vector Representation: Prediction using language models starts with encoding inputs into a vector space. In our case, we embed test cases into a suitable vector representation using CodeBERT and UniXcoder, respectively.

Both CodeBERT and UniXcoder take the test case code as input and convert it into a sequence of tokens. These tokens are then transformed into integer vector representations (embeddings). The test case code is tokenized using Byte−Pair Encoding (BPE) [43], which follows a segmentation algorithm that splits words into a sequence of sub-words.

Each token is then mapped to an index, based on the position and context of each word in a given input. These indices are then passed as input to the CodeBERT and UniXcoder models to generate a vector representation for each token as output. Finally, for a test case, all vectors are aggregated to generate a vector called `[CLS]` [12].

2) Fine-Tuning Code Models: As discussed earlier, we have fine-tuned CodeBERT and UniXcoder independently with two different techniques. Given our limited labeled training data, which is a common situation in our application context, few shot learning approaches [27], such as the Siamese Network architecture [44], are adequate candidate solutions. A Siamese Network trained with a specific loss function (*Triplet Loss Function* [45]) is designed to determine the similarity between inputs by learning an embedding space [46]—a high-dimensional vector space where inputs are represented as embeddings or vectors. In this space, similar inputs are positioned closer together, while dissimilar inputs are placed farther apart. Similar to existing work [23], we constructed a Siamese Network [23], [47], [48] with three identical sub-networks. Each sub-network is based on either CodeBERT or UniXcoder, essentially using the same model (weights and network parameters) three times within the network structure. The inputs to this Siamese Network are *anchor*, *positive* and *negative* tests, which are transformed by each sub-network of the Siamese Network into high-dimensional embeddings [46]. The anchor and positive tests, which belong to the same class, are projected into the embedding space such that their distance is minimized, while the distance between the anchor and the negative test, belonging to a different class, is maximized. This process ensures that similar inputs are grouped together in the vector space, facilitating tasks such as

[11] https://github.com/emobility-smart-charging/pull/42
[12] https://github.com/sofastack/sofa-boot/pull/1124

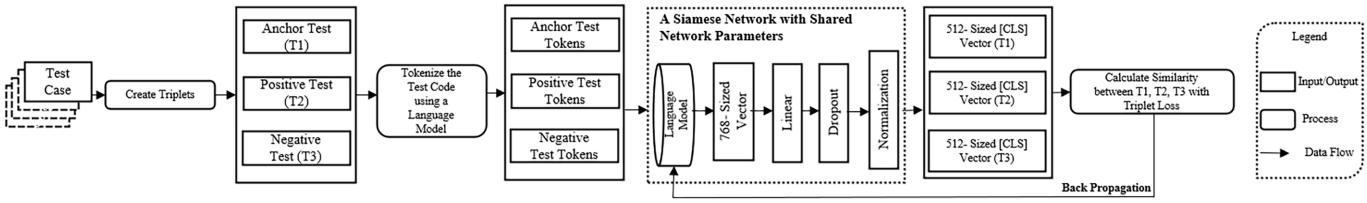


Fig. 3. Training of a Siamese network with a shared language model and network parameters.

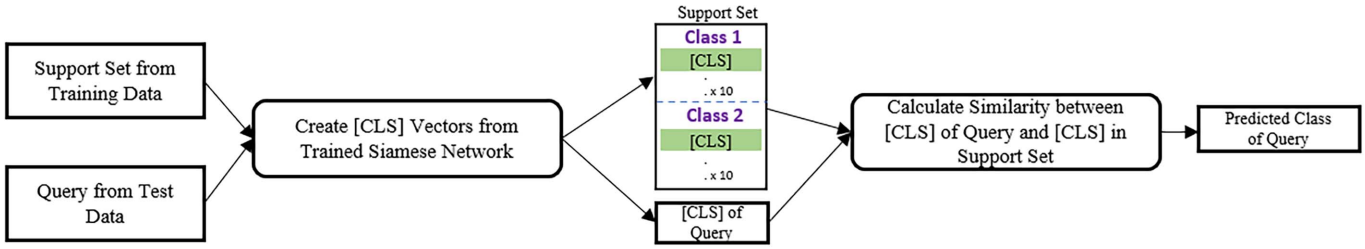


Fig. 4. Predicting the class of query test case using few shot learning and a trained Siamese network.

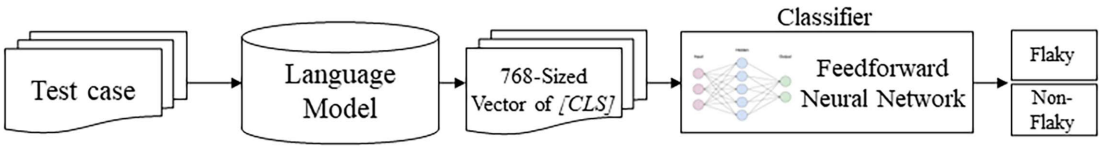


Fig. 5. Fine-tuning language model for classifying tests into different fix categories [12].

similarity comparison or classification based on learned similarities. During training, the triplet loss function fine tunes the embeddings by computing similarity scores, encouraging the network to improve its ability to distinguish flaky tests belonging to different categories. In our approach, given a category (e.g., *Change Assertion*), the anchor is a flaky test in the training dataset that belongs to the positive class of that category, e.g., it is labeled as *Change Assertion*. A positive sample is another flaky test in the training dataset that belongs to the same class as the anchor (e.g., labeled again as *Change Assertion*) and is considered similar to the anchor. The negative sample is another flaky test from the dataset that belongs to a different class than that of the anchor (e.g., NOT labeled as *Change Assertion*) and is considered dissimilar to the anchor. More details about the procedure to create these samples in our experiment will be provided in Section III.

We use the same architecture as Akli et al. [23] for each sub-network. First, it includes a linear layer of 512 neurons. To prevent model over-fitting, we then add a dropout layer to randomly eliminate some neurons from the network, by resetting their weights to zero during the training phase [49]. Lastly, we add a normalization layer before each sub-network that generates a `[CLS]` vector representation for the input test code of that sub-network. Fig. 3 depicts the architecture of a Siamese Network where anchor, positive and negative tests share a language model and other network parameters to update the model's weights based on the similarity of these tests. For training both CodeBERT and UniXcoder, we use the AdamW optimizer [50] with a learning rate of 10^{-5} and a batch size of two, due to available memory (8 gigabytes).

To evaluate the final trained model, following FSL, we create a support set of examples, for each prediction task. As a reminder, each prediction task here is a binary prediction of flaky tests indicating whether they belong to a fix category (positive class) or not (negative class). The support set includes a small number of examples for each class, from the training dataset. For every flaky test case in the support set of each class (positive and negative), we first calculate their `[CLS]` vector embeddings. We then calculate the mean of all test case vectors in that class, to represent the centroid of that support set class.

Each flaky test in the test data is referred to as a *query*. To infer the class of a query, from the test data, we first calculate the vector representation of the query using the trained model. Then using the cosine similarity of the query and the centroid of the classes, we determine to which class (positive or negative), the query belongs, i.e., the one with the highest similarity. Fig. 4 shows the process of predicting the class of a query using FSL.

Regarding our second method for fine-tuning CodeBERT and UniXcoder, we train a Feedforward Neural Network (FNN) to perform binary classification for each fix category. This process is depicted in Fig. 5 and was used by Flakify, which relied on CodeBERT [12]. The output of the language model, i.e., the `[CLS]` token which is the aggregated vector representation

token of all input tokens, is then fed as input to a trained FNN to classify tests into the fix categories. The FNN contains an *input* layer of 768 neurons, a *hidden* layer of 512 neurons, and an *output* layer with two neurons. Like previous work [12], we used ReLU [51] as an activation function, and a *dropout* layer [50]. Lastly, we used the *Softmax* function to compute the probability for a flaky test to belong to a given fix category.

C. Repairing Flaky Tests Using GPT and Our Proposed Fix Categories

Large Language Models (LLMs) such as GPT have been successfully applied to many automated software engineering tasks in the past [52], [53], including automated program repair [54], [55], [56] and [57]. Therefore, as part of our study, we are interested to see (a) how effectively an off-the-shelf GPT can generate fixes for our flaky tests and (b) how our classification of fixes can potentially help GPT to better fix flakiness.

We use GPT-3.5 Turbo, with 175 billion parameters and a context window of 16,385 tokens, designed to accommodate long inputs (in our case full test cases). This GPT version was released on November 06, 2023 and has shown superior performance for diverse code-related tasks compared to prior versions. As per OpenAI's official website [58], this model has been trained on data up to September 2021.

GPT-4, which was unveiled early 2023, features 1.7 trillion parameters, significantly surpassing GPT-3.5 in performance. However, the latest GPT-4's training dataset extends up to December 2023. Further, there is another version of GPT-4 that is trained up to April 2023. Given that a majority of flaky tests in our dataset were posted on the public repository before these two dates, we chose not to use GPT-4 as it could lead to data leaks. Though there are more recent test case fixes in our dataset, most of them are not yet accepted changes and we only consider fixes that are approved by the developers.

1) GPT Prompt for Fixing Flaky Tests: To evaluate the impact of predicted fix categories on GPT's repair capability, we incorporate the predicted category alongside the flaky test code as input query to GPT (the prompt). We use two different methods to build the prompt. The first simply provides flaky test code with its corresponding fix category label, while the second employs in-context learning. Influenced by recent research on prompt optimization for GPT [30], in-context learning provides multiple examples of inputs and their expected outputs, as the context. Since fine-tuning a LLM is expensive, with in-context learning, the LLM learns at inference time [59] on how to optimize its output for a particular problem domain (characterized by the examples in the context).

Section III-C3 shows our both prompt templates. In our current implementation of in-context learning, a prompt contains the flaky test requiring repair, its predicted category, and some sample flaky tests from the same category in the training set, accompanied by their respective fixes. This methodology enriches the model's output (the fix) by exposing it to prevalent resolution strategies applied to flaky tests within a specific category. Future investigations could delve into further optimizing these in-context learning prompts [60] to get the best results possible from GPT.

III. Validation

The goal of this study is to predict fix categories for a flaky test to help generate a fully repaired version of the test. This section validates our approach (See Section II) towards that goal by first introducing our research questions and explaining the data collection procedures. Then it discusses the experimental design and report the results.

A. Research Questions

In this study, we address the following research questions:

RQ1: How accurate is the proposed automated flaky test fix category labeling procedure?

Motivation: Given that there is no existing dataset of flaky tests that is labeled by fix categories, in this RQ, we validate our proposed procedure to automatically label a flaky test given its fix. If accurately automated, such a procedure is quite useful since it can convert any flaky test dataset that includes fixes, to an appropriate training set for building prediction models, such as those used in RQ2. In practice, a customized training set (from local projects) is often necessary to capture domain knowledge, either to train new models from scratch or fine-tune pre-trained models, which further justifies the need for an automated labeling technique.

RQ2: How effective are different language models, i.e., CodeBERT and UniXcoder with and without FSL, in predicting the fix category, given flaky test code?

Motivation: Most existing prediction models related to flaky tests try to predict flakiness. FlakyCat [23] is, however, the only approach that classifies flaky tests according to their root cause using CodeBERT and FSL. Identifying the root cause alone, however, does not directly aid testers in resolving flakiness. For instance, one suggested root cause in FlakyCat [23] is *Unordered Collections*, highlighting that it causes flakiness. However, the proposed solution categories in [20] maps this cause to the *Guarantee Order* fix category without providing concrete guidance on how to ensure such order. In contrast, our defined fix categories offer a one-to-one mapping of specific fixes to individual test cases. For instance, concerning *Unordered Collections* cause, Table II presents multiple fix categories such as *Reorder Data*, *Change data structure* or *Reorder Parameters*. Then depending on the test code one of these concrete fixes is going to be predicted as the fix category. This approach provides testers with more precise solutions rather than a broad label like *Guarantee Order*.

This practical difference between a fix and a root-cause, led us to investigate alternative approaches for classifying the tests according to their required fix category, given that these categories are based on actual fix patterns. In this RQ, we explore two code models as representatives of the state of the art. We also study the effect of FSL with each model, since FSL is usually recommended when the training data set is small, like in our case.

RQ3: How effective is GPT-3.5 Turbo in repairing a given flaky test, with and without information regarding its fix category and fix examples?

Motivation: Given a fix category, the next question is to check to what extent having that information helps in fixing flakiness. To answer that, we focus on a fully automated flaky test repair approach and want to assess whether knowing the flaky test category help obtain better repairs. To automate the repair process, we chose to rely on LLMs that are pre-trained on an extensive corpus coming from various resources, enabling them to generate complete code without requiring fine-tuning.

The recent development of LLMs with 175 billion parameters for GPT-3.5 Turbo and 1.7 trillion parameters for GPT-4 [58], trained over a dataset size of 45TB [61] offers significant potential for automating tasks related to source code, including automated program repair [57]. Thus RQ3 investigates whether the fix categories predicted in RQ2 enhance the performance of GPT models in repairing flaky test cases and to what extent we can achieve correct repairs. Additionally, we use in-context learning to understand how the GPT model's ability to fix issues improves when given examples of a specific type of flaky fix alongside our suggested fix category label.

B. Dataset and Data Augmentation

To create a labeled dataset of flaky test fix categories for our study, we first analyzed various datasets of flaky tests that include fixes. Then we picked the most suited dataset and augmented it with our automatically generated labels.

1) Existing Datasets: There is no publicly available dataset of flaky tests that have been labeled based on the nature of their fix. Most existing datasets for flaky tests, such as FlakeFlagger [14], simply contain binary labels indicating their flakiness. The dataset presented in the most related work, FlakyCat [23], assigns a label to each flaky test based on the category of flakiness causes, not their fixes. Therefore, we needed to build our own labeled dataset out of an unlabeled one that consists of both flaky tests and their fixes. We also required a dataset with a sufficient variety of flaky test fixes. Previous research on flaky test repair [62] has relied on datasets of limited size and scope. For instance, the Flex [62] dataset comprises only 35 flaky tests. Eck et al. proposed a study based on the Mozilla database of flaky tests [63] where they analyzed 200 flaky tests repaired by developers. Another example is the ShowFlakes dataset [20] that includes 75 commits from 31 Python projects. However, it only includes 20 flaky tests where flakiness can be removed by only changing the test code. The iPFlakies and iFixFlakies datasets [35], [64] are larger in size compared to previous datasets, but the root cause of flakiness in these datasets is related to the order in which the tests are executed, not the test code logic.

The largest publicly available and relevant dataset is the International Dataset of Flaky Tests (IDoFT)[13], an open-source dataset that is continuously updated. In this research, we collected a Java dataset of flaky tests from IDoFT until October 23, 2023. The dataset originally contained 5,500 tests, including 128 duplicates. After removing duplicates, we were left with 5,372 tests. We then filtered this set to include only those flaky tests for which the fixes were accepted, meaning that the pull requests (PRs) addressing these flaky tests on GitHub were approved by the original developers, thus resulting in 788 tests. From this subset of flaky tests, we analyzed the diff of the original flaky tests and their developer repairs. We removed 226 tests that had repairs exclusively in the code under test (CUT) with no changes to the test code, leaving 562 tests that were fixed by modifying the test code. As explained in the Introduction section as well, the flaky tests that can be fixed in a black-box manner are in the minority, but they are still important since testers and testing tools can fix them even without any access to production code, which is typical for some QA teams. The remaining tests were resolved through test configuration adjustments, changes in test case execution order, fixes implemented in the production code, or are pending in open pull requests (awaiting approval, rejection, or being deliberately skipped by the original developers). More details about this can be found online on the IDoft Repository. Consequently, our final dataset comprises 562 flaky tests and their corresponding resolutions from the IDoFT Java dataset. Notably, this dataset represents the largest available repository of flaky tests that are fixable through modifications of the test code. These tests originate from 123 distinct projects, offering a broad spectrum of test cases, which greatly benefits the generalizability of our predictive models.

2) Labeled Dataset of Flaky Test Fix Categories: To create the labeled dataset out of the IDoFT dataset, we followed the procedure discussed in Section II and labeled each flaky test automatically. Our dataset is publicly available in our replication package[14]. The resulting labeled dataset is unbalanced with respect to the distribution of tests across fix categories. Though, in our prediction phase, to handle the lack of data, we apply Few-Shot learning, to effectively predict the fix categories will require a minimum number of test cases per category. Therefore, we selected the categories with at least 30 tests as our training set. As a result, our labeled dataset consists of the top nine fix categories outlined in Table II for training our model. Since a flaky test may be associated with multiple fix categories, we have transformed our multi-label dataset into nine distinct bi-labeled sets, one per category. In each of the nine sets, the positive class includes tests that belong to the corresponding fix category and the negative class includes all the other tests. This allows us to treat the problem of predicting each label (category) as a separate binary classification problem.

3) Data Augmentation for FSL: We investigated code models with and without using FSL. Without FSL, we fine-tuned the code models with an FNN as discussed in Section II. For this, we used random oversampling [12], [65], which adds random copies of the minority class to our training dataset, which otherwise would be imbalanced (fewer positive examples).

In our fine-tuning approach, inspired by FlakyCat [23], we addressed limited training data by employing data augmentation

[13]https://mir.cs.illinois.edu/flakytests

[14]https://figshare.com/s/47f0fb6207ac3f9e2351

techniques [66], [67] along with a Siamese Network, triplet loss function, and FSL. Specifically, we synthesize additional valid triplets for training, by creating <anchor, positive, and negative> tests for each flaky test in the training data, thus increasing the training samples. For example, if the original training dataset has 20 samples, data augmentation will potentially create 20 new positive samples and 20 new negative samples, resulting in a total of 60 training samples. For each triplet per category, the positive test must belong to the positive class (same as the anchor test), while the negative test must belong to the negative class. To synthesize additional positive and negative tests, we mutate the test code by replacing the test names and changing the values of variables (integers, strings, and Boolean values) with randomly generated values. For example, for a test case in the training set named *postFeedSerializationTest*, we create a new test case (and label it with the same class label) and call it *postFeedSerializationTest_xacu4*. Then we change the value of an integer variable 'access point' from '1' to '8967', randomly. Also, the string value is randomly replaced from 'AMAZON-US-HQ' to 'AMAZON-"RZjqJ"-HQ'.

We automatically verify the created pairs, to ensure no duplicate tests are generated and to retain all flakiness-related statements unchanged. We thus produce enough samples to form the triplets (a set of anchor, positive and negative tests) required for this fine-tuning procedure.

4) Data Selection for Evaluation With GPT Model: In RQ3, we selected 562 tests to assess how well the GPT model deal with our flaky tests with and without fix category. Since the GPT 3.5 model was trained on collected data up to September 2021, we only included tests that were fixed or added after that date. Including older tests could be considered data leakage, since these tests are in the public domain (GitHub) and there is a high chance that they are in the model's training set. This exclusion criterion resulted in 181 tests from 61 distinct projects which included at least one test from each of the nine fix categories.

5) Data Selection for Execution of Repaired Flaky Tests: To gain further insights into the GPT-generated repaired flaky tests, we conducted executions of selected tests. Our dataset comprises 181 tests from 61 distinct projects. Among these projects, we selected projects where at least five flaky test cases were present, which resulted in nine projects. Out of these nine projects, the test cases of four projects could not be executed due to missing dependencies, resulting in build failures, or the unavailability of certain files linked to flaky tests mentioned in the IDOFT dataset, either due to merged commits or non-existent files in our local system. Consequently, from these 5 successfully executed projects, we collected results of all their flaky tests (35 test cases). We have provided details of these projects in our replication package.

C. Experimental Design

This subsection covers the baselines, code model training and testing, GPT model's prompt template, and execution of the repaired flaky tests.

1) Baseline: Automatically defining different categories of flaky test fixes using only test code and then labeling flaky tests accordingly along with generating a full repair of flaky tests is a task that has not yet been studied before. Previous work has either analyzed the causes of flakiness [23] or manually identified the fix for flaky tests [20]. To design our experiment, we only focused on code models for prediction, as explained in Section II-B. As our technique relies exclusively on test code, we cannot compare it with traditional machine-learning classifiers, which require a pre-defined set of features such as execution history, coverage information in production code, and static production code features [12]. Alternatively, only relying on predefined static features from test code is challenging. These features may not capture all the semantics of the code that contribute towards flakiness and may not be good indicators to predict fix categories. Lastly, defining good features, often referred to as feature engineering [67], is not an easy task. In contrast, we aim to develop a solution that is practical where testers just pass the test code directly to a model as input and get information regarding their fix category, rather than going through the process of defining features.

We excluded traditional sequence learning models as baselines, such as the basic RNN or LSTM models, since they are not pre-trained like code models. Hence, they require large datasets to train their many parameters and avoid over-fitting [68], [69].

Therefore, we evaluated our idea using two SOTA code models, namely CodeBERT and UniXcoder, both with and without FSL. FSL has been recommended in previous work for small datasets (e.g., for flakiness root-cause prediction [23]), but has not been compared with code models without FSL. Thus we also included this experiment.

2) Training and Testing Prediction Models: To evaluate the performance of our code models for predicting the fix categories, we employed a stratified 4-fold cross-validation procedure to ensure unbiased train-test splitting. This procedure involves dividing the dataset into four equal parts, with three parts allocated for training the model and one part held out as the test set. Moreover, to prevent over-fitting and under-fitting during the model's training, we employed a validation split, which consists of 30% of the training data. This validation dataset is used for fine-tuning code models using FNN and a Siamese Network. We deliberately did not use a higher number of folds since for smaller categories that would not leave enough samples in the train and test sets. For instance, the *Change Data Format* fix category has only thirty-three positive examples, which would lead to only four positive examples in each fold of a 10-fold cross-validation.

3) Prompt Design: To get the fixed flaky test code from GPT, we use three different types of prompts, as follows: (1) we pass the flaky test code in the prompt to the model and ask it to generate its fix as shown in Fig. 6, (2) we provide both the flaky test code and the fix category label we predicted for that flaky test. We then ask the GPT model to generate a fix as shown in Fig. 7, and (3) we extend the content of the second type by also passing a few examples to the model as shown in Fig. 8. This type of prompting aims to assess GPT's performance with

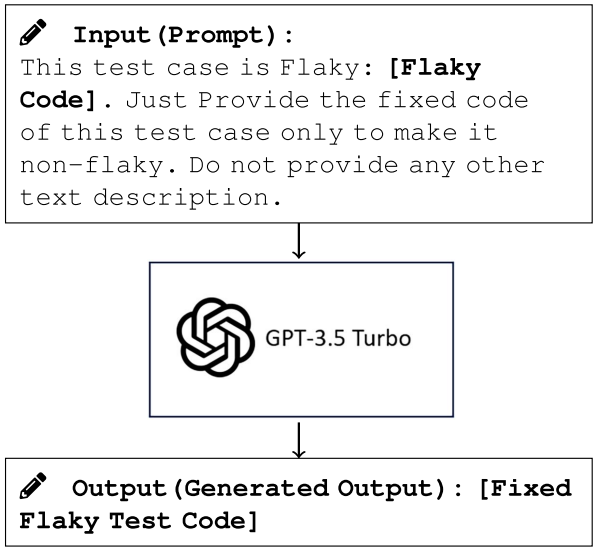


Fig. 6. Flaky test fix using GPT-3.5 Turbo without fix category label.

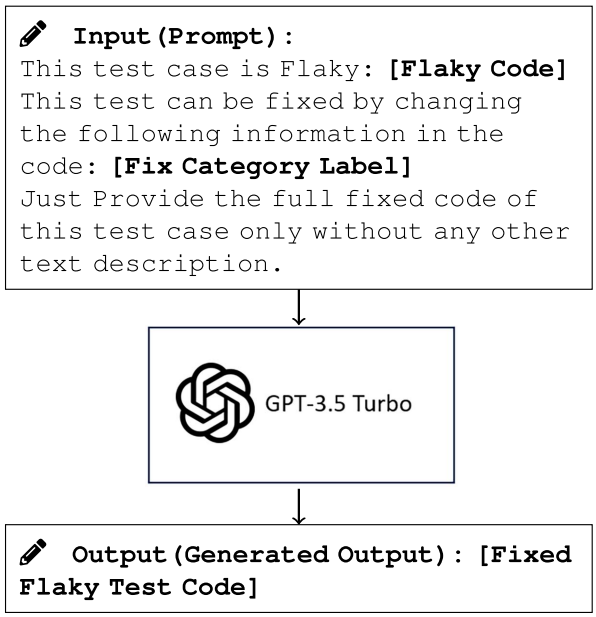


Fig. 7. Flaky test fix using GPT-3.5 Turbo with fix category label.

in-context learning. We focused on three fix categories comprising more than 30 instances (to have large enough sample sizes to make meaningful conclusions) within our dataset of 181 tests: *Change Assertion* (64 examples), *Change Data Structure* (41 examples), and *Change Condition* (41 examples). This resulted in a dataset of 146 tests. From these, we randomly selected 5 examples from each category for in-context learning, leaving a total of 131 tests for our in-context learning experiments.

We used these selected examples of flaky tests, along with their respective fixes and fix category labels, as prompts for the GPT model. Our prompt requested the model to provide a fix for a new flaky test based on a given fix category label, utilizing information derived from these example tests and their

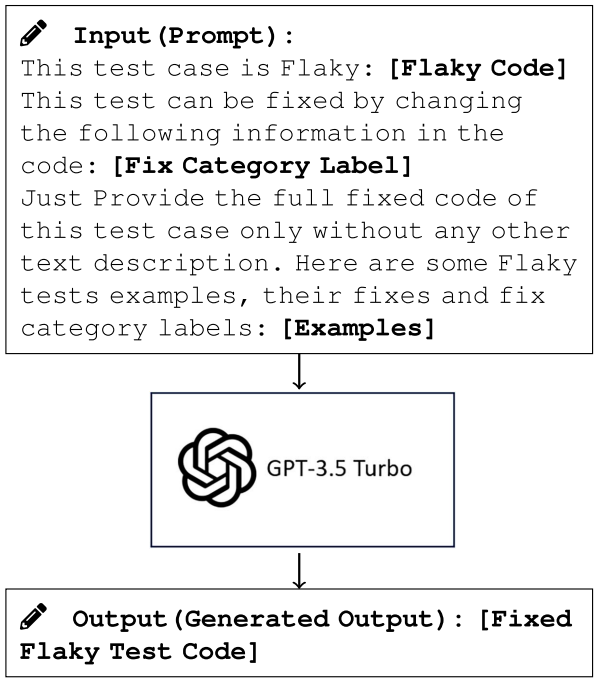


Fig. 8. Flaky test fix using GPT-3.5 Turbo with in-context learning.

associated fix category labels. Our prompt explicitly requests the model to provide only the fixed test code, excluding any additional textual descriptions. This approach streamlines the output, ensuring that the model generates the test code directly without requiring post-processing. In the future, we plan to further test our GPT model using a larger number of examples of flaky tests. This expansion aims to evaluate and enhance the model's capabilities in handling varied scenarios.

4) Test Case Execution: To verify the automatically generated repairs, as described in Section III-B5, we ran 35 tests obtained from 5 projects that could be executed. To do so, initially, we executed the original developer-repaired versions of the 35 tests, 10 times each, to help verify the approved fix was indeed correct. Then, we executed the LLM-generated versions of these tests, 10 times each as well. Given that a test may remain flaky even after repair if the correction is inadequate, multiple runs were necessary to ascertain consistent outcomes across executions. That is, a passing or failing test should consistently pass or fail, respectively, in all 10 runs. We have included detailed instructions for executing these tests, along with the corresponding results, within our replication package.

5) Evaluation Metrics: To assess the effectiveness of our classifier, we employed standard evaluation metrics, including precision, recall, and F1-Score, which have been widely used in previous studies of ML-based solutions for flaky tests [12], [14], [15], [17], [23]. In our case, precision is of utmost importance, and we emphasize its role in our results. In practical terms, once a fix category is predicted for a flaky test, such as *Change Assertion*, we want testers to confidently proceed with fixing the issue by focusing on the code statements matching the category, e.g., by modifying the assertion type used. Accurately

predicting the correct fix category is therefore critical since attempting to fix a test incorrectly can result in wasted time and resources [70] and may even cause additional failures in the test suite [6], [71].

We used Fisher's Exact test [72] to assess whether there was a significant difference in performance among the four prediction methods: fine-tuned CodeBERT and UniXcoder with and without FSL, across all nine fix categories. Furthermore, in RQ1, we used the Kappa score [36] to evaluate the level of agreement between the researchers who initially labeled the dataset. By using the Kappa score, we were able to assess the reliability and consistency of the initial labels across different categories of fix, despite their varying sample sizes. Additionally, in RQ3, to evaluate how useful our classification is for improving an automated repair model (i.e., GPT in this case), we have used several metrics from the BLEU score family [73] (i.e., Corpus BLEU, Sentence BLEU, and CodeBLEU), which are widely used metrics in automated code generation and repair studies [29], [74], [75], [76].

Particularly, CodeBLEU is crucial as it captures both the semantics and syntax of the code, providing a more precise evaluation of the generated code. Unlike the traditional BLEU score, which assesses the similarity between candidate and reference texts, based solely on sequences of n contiguous tokens (n-grams) overlapping, CodeBLEU incorporates additional components to account for the specific code characteristics. It is defined as a weighted combination of four parts [77] as shown in the below equation.

$$\begin{aligned}\text{CodeBLEU} = \alpha \cdot \text{BLEU} + \beta \cdot \text{BLEU}_{\text{weight}} \\ + \gamma \cdot \text{Match}_{\text{ast}} + \delta \cdot \text{Match}_{\text{df}}\end{aligned} \tag{1}$$

where:

- BLEU represents the standard BLEU score, measuring the overlap of n-grams between the generated and reference code.
- $\text{BLEU}_{\text{weight}}$ denotes the weighted n-gram matching, assigning varying weights to n-grams to emphasize important code keywords like *int*, *char*, *public*, and others. Different weights are assigned to different n-grams, with keywords often receiving higher weights.
- $\text{Match}_{\text{ast}}$ signifies the syntactic AST match, focusing on the syntactic structure of the generated code.
- Match_{df} assesses the semantic similarity between generated and reference code by examining their data-flow representations. Code is represented as a graph where nodes represent variables, and edges indicate how variable values flow. Unlike Abstract Syntax Trees (AST), data-flow graphs illustrate how variables interact within the code. We are thus able to measure the semantic match between the candidate and reference code.

The parameters α, β, γ, and δ in Equation 1 represent the weights of the above-mentioned four parts in the CodeBLEU score. We have used a value of 0.25 for all four weights, as used by Ren et al. [77]. These four components collectively evaluate both the grammatical correctness and logical correctness of the generated code, making CodeBLEU a comprehensive metric for code evaluation.

Beside syntactic similarity metrics like CodeBLEU, we also executed a subset of repaired tests to assess the correctness of GPT-repaired flaky tests. Analysing the execution results, we calculated the passing rate of the repaired tests. Then using bootstrapping [78] with a 95% confidence level, a statistical method for estimating properties of a large sample from repeated sampling (with replacement) from a small sample, we established lower and upper bound percentages of passing test cases within the entire flaky test set, including non-executable test cases. Additionally, we used a logistic regression [79] model to predict the pass rates of non-executable tests based on their CodeBLEU scores in various prompt settings.

Furthermore, for passing tests, we manually ensured they did not introduce regressions regarding the test class's bug-finding capability. While tests with a CodeBLEU score of 100% didn't need equivalence checks, for those with scores below that, we manually verified behavioral equivalence with the ground truth. This was possible given the relatively low complexity of the test cases.

Regarding failing tests, we calculated the Levenshtein Edit Distance [80] between the generated and actual fixes. This is a measure of similarity between two strings that determines the minimum number of token edits needed to transform one into the other. We can then compute the number and percentage of tokens needing change to achieve perfect alignment with the original developer-repaired test.

Lastly, We employed the Wilcoxon signed-rank [81] test to gauge the significance in performance differences of the GPT model across various input prompts.

D. Results

1) RQ1 Results: To start, 100 test cases were randomly selected and independently labeled by two researchers (that are among the co-authors of this research). The Kappa score was found to be 0.65, indicating a moderate level of agreement between the two raters. Disagreements between them were eventually resolved with multiple rounds of discussions to finalize the fix categories that are used by the automated labeling tool.

To evaluate the labeling accuracy of our automated tool, we took another random set of 200 flaky tests and obtained their fix category labels from the tool. The same two researchers then manually labeled these tests, enabling us to calculate the accuracy of our automated labeling tool as shown in Fig. 2. We found that the tool had an accuracy of 98.5%, with only three tests being mislabeled with multiple fix categories, including the correct one.

RQ1 summary: Our proposed automated labeling system for flaky tests was evaluated and yielded an accuracy of 98.5% in correctly labeling a test set of 200 tests with various fix categories. Having an automated and accurate labeling tool is essential for this study, future research, and in practice when building such prediction models in context.

TABLE III
COMPARISON OF THE PREDICTION RESULTS FOR EACH FIX CATEGORY USING CODEBERT (CB) AND UNIXCODER (UC), BOTH WITH AND WITHOUT FSL. THE HIGHEST VALUE PER METRIC FOR EACH FIX CATEGORY IS HIGHLIGHTED

Metric	**Precision** (%)				**Recall** (%)				**F1** (%)			
Category	CB	CB with FSL	UC	UC with FSL	CB	CB with FSL	UC	UC with FSL	CB	CB with FSL	UC	UC with FSL
CA	88	86	**96**	93	88	**86**	**86**	85	88	86	**91**	89
CDS	86	76	**89**	85	86	71	**87**	81	87	74	**88**	83
HE	97	95	**98**	95	87	88	89	**90**	92	91	**93**	**93**
RV	88	91	94	**96**	**98**	95	97	96	93	93	**96**	**96**
CC	91	84	**87**	74	84	70	**87**	70	**87**	76	**87**	72
RD	88	80	**96**	90	82	**88**	72	**88**	85	84	82	**89**
CDF	96	**97**	90	90	93	**97**	94	90	96	**97**	93	95
MC	**83**	79	82	76	**67**	61	62	61	**74**	**70**	70	68
RP	53	48	**91**	88	**94**	88	**91**	82	64	68	**91**	85

2) RQ2 Results: Table III presents the prediction results for all nine categories of flaky test fixes using four distinct approaches, namely CodeBERT and UniXcoder, fine-tuned with Siamese Networks or FNN. In the former case, we evaluated both language models using Few-Shot Learning (FSL).

UniXcoder without FSL demonstrated superior precision and similar recall in the *Change Assertion*, *Change Data Structure*, *Handle Exception*, *Change Condition*, and *Reorder Parameters* categories, with precision of 96%, 89%, 98%, 87%, and 91%, respectively, outperforming the other three approaches. For *Reorder Data*, again UniXcoder without FSL achieves the highest precision score of 96%. However, when incorporating FSL, UniXcoder demonstrates a superior recall of 88%. Furthermore, CodeBERT without FSL showed slightly better results for the *Miscellaneous* category, with 83% precision and 67% recall, though the difference with UniXcoder is not significant. For the *Reset Variable* category, UniXcoder with FSL yielded high accuracy with 96% precision and 97% recall rates, while CodeBERT without FSL achieved a slightly higher recall of 98% albeit with a low precision of 88%. Finally, for *Change Data Format*, CodeBERT with FSL outperformed all other approaches with a 97% precision and recall. However, again the difference with UniXcoder is not significant.

The *Change Data Format*, *Handle Exception*, *Reset Variable*, *Reorder Data* and *Change Assertion* categories were the easiest for all four approaches to classify, with an average precision score (across all approaches) of 93%, 96%, 92%, 88%, and 90%, respectively. This is likely due to the high frequency of common keywords in test code for exception handling, assert statements, resetting variables, and reordering data (with keywords like "sort" and "order by"). In the case of the *Change Data Format* category, the model looks for a long complicated string that is being passed to a function. These strings are easier for the model to identify because we observed that all the flaky tests matching this category contain a long complicated string that needs to be updated to remove flakiness. All four models demonstrate strong performance across most fix categories, exhibiting high recall and precision. However, in the Miscellaneous Category, their performance notably decreases, averaging at 62%. This lower performance may be attributed to the absence of specific common patterns or code keywords across all tests within this category, which cannot be expected to be homogeneous. Moreover, in the Reorder Parameter Category, CodeBERT, with and without FSL, exhibits significantly lower precision compared to UniXcoder. Further details about the models' misclassifications are provided in Section IV.

Overall, the results presented in Table III suggest that, for most of the fix categories, UniXcoder outperforms CodeBERT significantly, based on a Fisher's Exact test with $\alpha = 0.05$. However, FSL did not significantly improve UniXcoder's performance, which is probably due to the limited dataset with only a small number of positive instances for most fix categories, which hindered fine-tuning.

RQ2 summary: UniXcoder outperformed CodeBERT for predicting most of the fix categories, with higher precision and recall. The use of FSL does not significantly improve the prediction accuracy of UniXcoder.

3) RQ3 Results: Table IV reports on the evaluation results generated by GPT 3.5 Turbo, which processes a flaky test and provides its corrected version, with and without our predicted fix category labels as input. The assessment metrics include median and mean scores for Corpus BLEU, Sentence BLEU, and CodeBLEU across the 181 flaky tests in the test set. The inclusion of our predicted fix category labels, in the prompt, increases the median of all three metrics by 7%, 6%, and 4% for Corpus, Sentence, and CodeBLEU, respectively. The improvements are statistically significant based on non-parametric Wilcoxon signed-rank tests at a significance level of $\alpha = 0.05$, with a p-value below 0.0001 in all three cases.

TABLE IV
EVALUATION OF THE GPT-3.5 GENERATED FLAKY TESTS USING BLEU SCORES ACROSS ALL CATEGORIES FOR 181 FLAKY TESTS

Metrics	Prompts With Labels		Prompts Without Labels	
	Mean	Median	Mean	Median
Corpus BLEU	76	82	69	75
Sentence BLEU	77	82	70	76
CodeBLEU	81	85	75	81

TABLE V
MEDIAN AND MEAN CODEBLEU SCORE PER FIX CATEGORY, WITH AND WITHOUT PROVIDING THE FIX CATEGORY LABELS IN THE GPT PROMPT

Fix Categories	CodeBLEU Score				Count
	Prompts With Labels		Prompts Without Labels		
	Mean	Median	Mean	Median	
Change Assertion	80	85	76	79	64
Change Condition	87	90	84	89	41
Change Data Structure	87	81	72	82	41
Misc	76	83	75	82	22
Change Data Format	77	80	60	58	13
Reorder Parameters	86	86	77	83	10
Reorder Data	71	73	69	73	5
Handle Exceptions	66	73	59	60	7
Reset Variable	95	95	95	95	1

This improvement is attributed to the additional information provided to the GPT model, aiding it in precisely identifying and addressing the issues causing test flakiness. This finding also indirectly suggests that our predicted fix categories are well defined and may effectively help testers identify the specific code segments that require modification to mitigate flakiness. In the rest of this RQ, we only report CodeBLEU scores, since they are designed to more directly assess the semantics and syntax of code rather than natural language text, thus providing a more accurate evaluation of generated test code.

Table V shows median and mean CodeBLEU scores per fix category, with and without using label information. As we can see, in most categories, the labels improve the mean and median CodeBLEU scores. The improvements per category range from 0% to 17% for the mean and from -1% to 22% for the median CodeBLEU. Smaller improvements are either in categories with very few samples (e.g., Reset Variables with only one sample) or when the results are already quite high (e.g., Change Condition with a median CodeBLEU of 89% without labels). The only category where the Median results slightly dropped (1%) after providing the labels is the Misc category. This is most likely because the *Misc* label does not offer practical information on how to resolve the flakiness.

Now to expand the analysis to the third prompting approach (in-context learning), Table VI compares the three approaches for the *Change Assertion*, *Change Data Structure*, and *Change Condition* fix categories. As explained, we only look at these categories since after removing 5 samples per category to be included in the context, only these categories have enough samples to do a proper analysis and run statistical tests.

Looking at the table, we see that, for the *Change Data Structure* category, the mean and median CodeBLEU scores increase from 80% to 86% and 87% to 94%, respectively, when we use in-context learning versus simply providing the labels. Similarly, for the *Change Condition* category, mean and median CodeBLEU scores increase from 88% to 92% and 90% to 97%, respectively. However, in the case of the *Change Assertion* category, there is no discernible improvement in the model's performance after exposure to a limited number of examples. This could be attributed to the variability of assertion statements within the test code, suggesting that multiple approaches may

TABLE VI
COMPARISON OF MEDIAN AND MEAN CODEBLEU SCORES FOR DIFFERENT FIX CATEGORIES USING FIX CATEGORY LABELS AND IN-CONTEXT LEARNING

Fix Categories	CodeBLEU Score						
	Prompts Without Label		Prompts With Label		Prompts with In-Context Learning		Count
	Mean	Median	Mean	Median	Mean	Median	
Change Assertion	77	80	79	84	79	85	59
Change Data Structure	75	81	80	87	86	94	36
Change Condition	85	88	88	90	92	97	36

exist to fix assertions and resolve flakiness. It is plausible that the number and variety of examples provided was insufficient for the model to grasp the diverse spectrum of assertion-related fixes. Further, as depicted in the boxplots in Fig. 11, the overall distribution of the CodeBLEU scores shows significant variance for all types of prompts and the three fix categories. The low minimum values in all cases shows that there are flaky tests that are not fixed regardless of the prompting strategy. However, when looking at the CodeBLEU over all three categories for in-context learning, it is clear that, for the majority of the cases, the CodeBLEU scores are quite high. For example, even for the Change Assertion category, which has the lowest minimum scores, 75% of samples have CodeBLEUs higher than 55%, and for half of the test cases the CodeBLEU scores are greater than 85%.

Given the variance in CodeBLEU values per category, we again used the non-parametric Wilcoxon signed-rank test with a significance level of $\alpha = 0.05$ to make sure the improvements we see in medians are statistically significant as well. Our findings indicate that, when accounting for all three categories, both for prompts with label and without label, in-context learning makes a significant difference, with p-values below the significance threshold $p < 0.001$. However, looking at individual categories, this difference for prompts with label is only significant for *Change Data Structure* and *Change Condition*, with p-values $p < 0.001$ and $p = 0.007$, respectively.

In future investigations, we intend to systematically explore the use of different prompts by augmenting the number and types of examples or experimenting with other prompt engineering strategies [75]. Further details regarding GPT's performance on individual fix categories are discussed in Section IV.

To gain deeper insights into the performance of GPT-repaired flaky tests, we conducted a series of evaluations based on test execution results. For the reasons explained in Section III-B5, we could only execute a total of 35 tests, each repeated 10 times, resulting in 24 passing and 11 failing tests, with consistent results across the 10 runs.

From the 35 executable tests, generated with labels and examples, for the three categories where the latter are available, we would like to first estimate the passing rate of fixed tests while accounting for the uncertainty associated with such a limited sample. Using Bootstrapping, we estimate a 95% confidence interval for the percentage of passing tests, that ranges from 51% to 83%. This interval is informative for testers, who do not have access to CodeBLEU scores in practical testing scenarios, and would like to know what to expect from FlakyFix. Based on these results, we conclude that FlakyFix provides an effective solution in a large proportion of cases.

Next we are interested in determining the relationship between execution outcomes and the corresponding CodeBLEU scores to ascertain the reliability of the CodeBLEU metric in assessing the generated test. Specifically, we expect that tests with higher CodeBLEU scores to have a higher likelihood of passing and vice-versa. Overall, the average score among passing and failing tests is 94% and 74%, respectively, suggesting that higher CodeBLEU scores are an indicator of passing tests. In particular, among the passing tests, 16 out of 24 achieved a perfect CodeBLEU score of 100%, indicating an exact match with the developer-repaired versions. Additionally, 22 tests scored above or equal to 80% on CodeBLEU. Conversely, among the 11 failed tests, only five tests achieved CodeBLEU scores above or equal to 80%. However, one passing test exhibited a notably lower CodeBLEU score of 36%, as illustrated in Fig. 12, showing an alternative but correct repair. This particular case perfectly illustrates the limitations of using CodeBLEU scores.

To quantitatively model the relationship between CodeBLEU scores and the test passing probabilities, we train a Logistic Regression (LR) [79] model on the 35 executable tests, using their pass/fail labels, achieving an accuracy of 80% (correctly predicted test cases). The choice of LR is due to the small sample and the fact that we have one explanatory variable, CodeBLEU, that is expected to have a monotonic relationship with the test passing probability. The model's goodness of fit, measured by Chi-Square test [82], yields a p-value of 0.010, which indicates a statistically significant relationship between CodeBLEU scores and the probability for a test to pass, despite the small size of the sample. The fitted LR curve between CodeBLEU scores and the probability for a test to pass is shown in Fig. 9, showing a sharp increase in passing probability beyond CodeBLEU scores of 50.

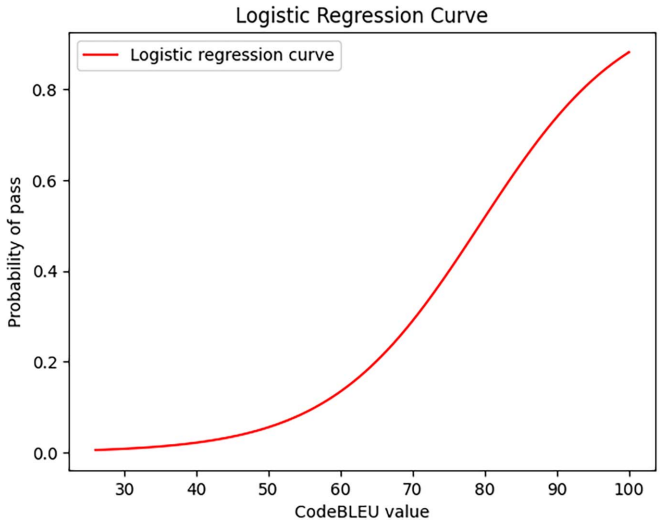


Fig. 9. The Logistic Regression Curve showing a relation between CodeBLEU scores and probability of passing.

To estimate the passing test rate among the non-executable tests, we apply this trained LR model to predict the outcomes of these tests. We calculate 95% confidence intervals for the predicted probabilities using standard errors derived via the delta method [83]. This method allows us to approximate the variance of the predicted probabilities by first calculating the standard errors on the logit scale and then transforming these to the probability scale. With this, we ensure that the confidence intervals account for the non-linear relationship between the logit and the predicted probability, giving a reliable measure of uncertainty around our predictions. According to our estimates, from the 181 test dataset, we obtain the following probability confidence intervals and passing test estimates:

- With No Label prompt: Confidence Interval: 0.44–0.50 — 80 (44%) tests to 91 (50%) tests are projected to pass.
- With Label prompt: Confidence Interval: 0.51–0.57 — 92 (51%) tests to 103 (57%) tests are projected to pass.

From the 131-test dataset (those that we can apply in-context learning on), we obtain the following results:

- With No Label prompt: Confidence Interval: 0.44–0.52 — 58 (44%) tests to 68 (52%) tests are projected to pass.
- With Label prompt: Confidence Interval: 0.50–0.58 — 66 (50%) tests to 76 (58%) tests are projected to pass.
- With in-context learning: Confidence Interval: 0.53–0.60 — 69 (53%) tests to 79 (60%) tests are projected to pass.

Tables VII and VIII provide the passing test estimates in terms of upper and lower bounds across different prompt settings, for the 181 and 131 test datasets, respectively.

From the above results, we can conclude that providing labels with the prompt to GPT does indeed make a difference in terms of passing tests. Overall, a reasonable proportion of generated tests with labels are predicted to pass, ranging approximately from 50% to 60% when accounting for uncertainty. Furthermore, for the 11 GPT-generated tests that failed upon execution, we calculated the edit distance as a surrogate measure to assess the manual effort required for fixing. The results revealed that, on average, 16% of the tokens needed to be replaced to convert a failed test into the corresponding ground truth. Fig. 10 depicts the trend between CodeBLEU scores and the percentage of token replacement. The scatter plot suggests that GPT-repaired tests with higher CodeBLEU scores tend to require fewer edits for repair, suggesting closer alignment with the developer-repaired versions. Lastly, to gain further insights into GPT's performance, we analyzed the fix categories of the 11 failed tests. Among these, six tests fell under the *Change Data Format* category. Of the remaining five, three were categorized under *Change Assertion* and two under *Change Data Structure*. The predominance of failed tests in the *Change Data Format* category can be attributed to the absence of in-context learning for this category, due to insufficient available instances as explained in Section III-C3. Our approach for this category relied solely on prompts with label information, affecting the quality of generations. However, as visible in Table VI, for the other three fix categories, the results indicate notable improvements in GPT's generation quality with the implementation of in-context learning. As mentioned earlier, all generated tests, along with their CodeBLEU scores and execution results, are provided in our replication package.

RQ3 summary: In all fix categories, either providing category labels or applying in-context learning significantly boosts the GPT model's ability to repair flaky tests. For two categories, where in-context learning is possible, it provides better results than prompts with labels. Furthermore, when providing labels in prompts and with in-context learning, when the latter is possible, we project that a large percentage ($> 50\%$) of repaired tests will pass. For the tests that fail, required changes are rather limited, with an average of 16% of the tokens to be changed in test code.

IV. Discussion

In this section, with qualitative analysis, we aim to ascertain not only theoretical soundness but also the practical utility of our solution in addressing the repair of flaky tests.

A. Qualitative Analysis of the Model's Performance

To gain a deeper understanding of the best model's performance, we have conducted a qualitative analysis of representative flaky tests from various fix categories.

1) Code Model's Prediction Results for Flaky Test Fix Category: In this section, we are analyzing sample test cases and their predictions using one approach, namely UniXcoder without FSL, since on mean it performs the best among the prediction models. We specifically target True Positives (TP) and a False Negatives (FN) from the same category, to provide the reader with insights regarding how these models can predict flaky fix categories. We also investigate False Positives (FP), since FP instances waste testers' time when they follow wrong suggestions and therefore directly affects the applicability of the proposed approach.

TABLE VII
PASSING ESTIMATES FROM THE 181 TEST DATASET FOR THE GPT-REPAIRED TESTS WITH AND WITHOUT PROVIDING THE FIX CATEGORY LABELS IN THE GPT PROMPT

Prompts	Lower Bound		Upper Bound	
	Passing Tests #	Passing %	Passing Tests #	Passing %
With No Label	80	44%	91	50%
With Label	92	51%	103	57%

TABLE VIII
PASSING ESTIMATES FOR THE GPT-REPAIRED TESTS USING FIX CATEGORY LABELS AND IN-CONTEXT LEARNING (FOR THE 131 TESTS)

Prompts	Lower Bound		Upper Bound	
	Passing Tests #	Passing %	Passing Tests #	Passing %
With No Label	58	44%	68	52%
With Label	66	50%	76	58%
In-Context Learning	69	53%	79	60%

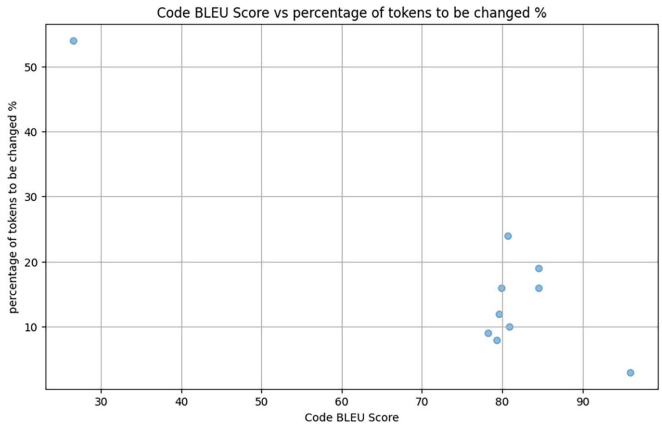
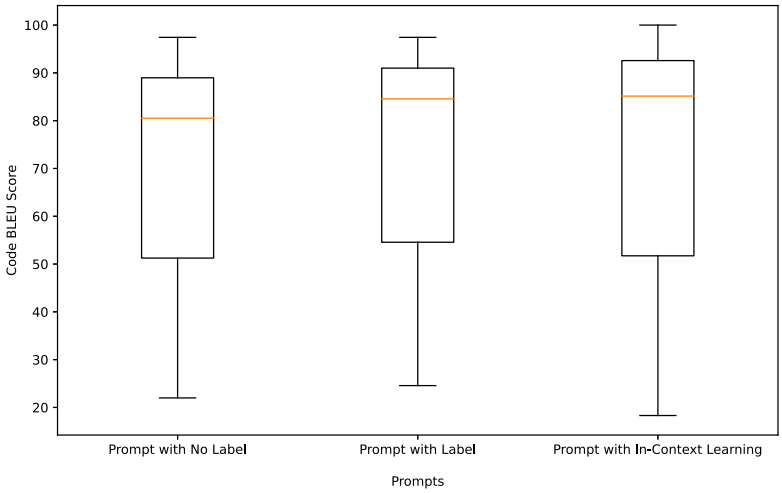


Fig. 10. Comparison of CodeBLEU Score with the percentage of tokens changed in failed GPT-generated tests.

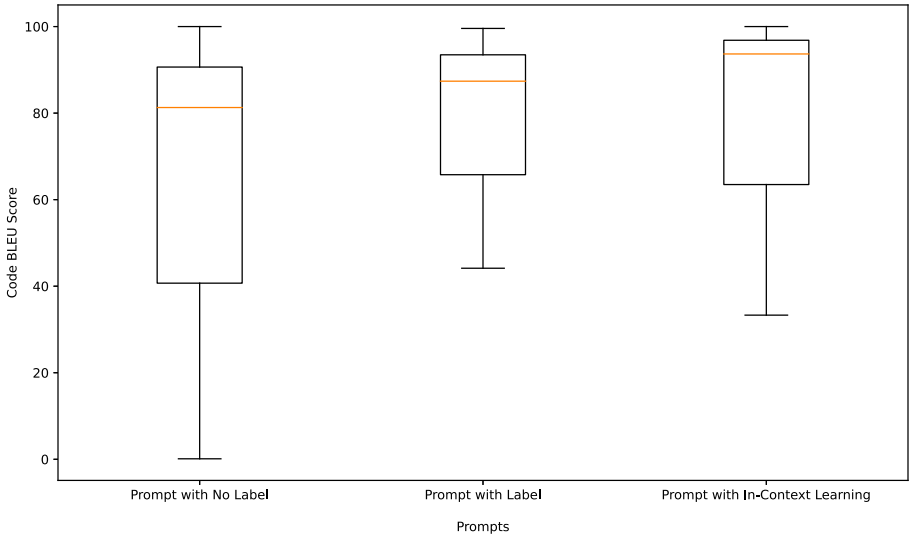
Fig. 13 shows a representative example test whose fix category is correctly predicted as *Change Data Structure* (TP). On line 2 of this test code, **HashMap** is used, which can cause flakiness, as discussed in Section II. Our model correctly predicts the category and suggests replacing **HashMap** with **LinkedHashMap**. Similarly, Fig. 14(a) shows another Flaky test whose fix category is correctly predicted as Handle Exception and Change Assertion (TP). Fig. 14(b) shows the developer fix of the same flaky test where a correct exception handling was added and *assertEquals* was changed to *assertJSONEqual* for correctly handling JSON objects. The test code shown in Fig. 15(a) is an example where our prediction model fails to correctly predict the fix category (FN) as *Handle Exception*. This test case differs from the typical patterns observed in that fix category because the keyword exception was already present in the method signature. Typically, exception handling is not part of flaky code in that category and an exception or try-catch is added in the code to resolve flakiness. However, in this case, it is the opposite and the pattern does not match other examples in the training set. When we examine the corrected test code shown in Fig. 15(b), we notice the exception had been removed as part of the fix. However, our automated labeling tool classified this test case as *handle exception* due to the presence of the 'exception' keyword.

In general, we do not have many examples of FP cases (13 tests on mean for all of the four approaches we have used), which is not surprising as it is one of the strengths of our proposed solution. One sample FP test code is shown in Fig. 16. This test was incorrectly classified as a *Change Assertion* since no change in the assertion type is needed. A potential reason for the misclassification can be related to the long string passed to the assert statement in the code. The pattern of using the long string is similar to cases where flakiness is addressed by changing an assertion to a more customized assertion with shorter inputs, for example using *AssertJsonStringEquals* to handle Json Strings. However, here the incorrect use of assert was not the cause for flakiness and thus was not changed. Although the recommendation was a FP, suggesting a Change in assertion might not be a bad idea after all, not as a fix but as refactoring to create more maintainable tests.

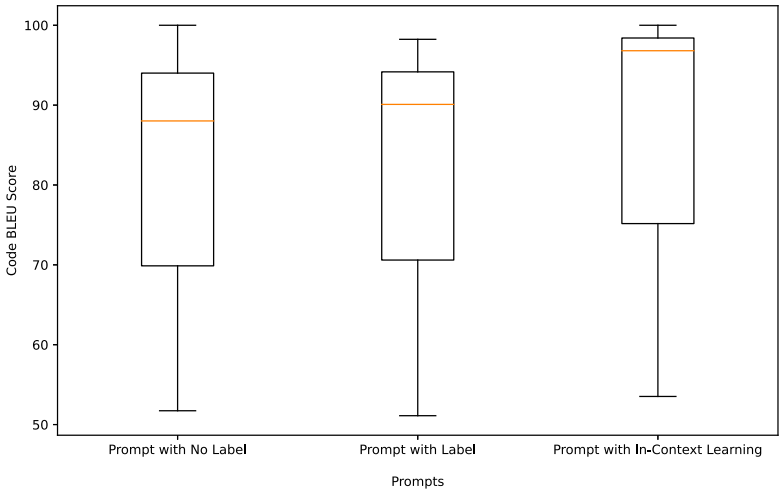
To summarize, we can conclude that the models are correctly learning the patterns that exist in the test code and, in most cases, FP or FN are simply due to lack of data, where without the actual fix or other sources of information about the bug, correct prediction is not possible.



(a) Change Assertion



(b) Change Data Structure



(c) Change Condition

Fig. 11. Distribution of CodeBLEU scores with respect to three prompts.

```
@Test
public void queryByEmptyKeySubObject() {
    JSONObject json = new JSONObject("{\":\"empty key of an object with an empty key\",\"subKey\":\"Some" +
                " other value\"}");
    JSONObject obj = (JSONObject) query("/obj/");
    assertTrue(json.similar(obj));
}
```

```
@Test
public void queryByEmptyKeySubObject() {
    JSONObject expectedJsonObject = new JSONObject();

    expectedJsonObject.put("", "empty key of an object with an empty key");
    expectedJsonObject.put("subKey", "Some other value");
    assertEquals(expectedJsonObject.toString(), query("/obj/").toString());
}
```

Fig. 12. Comparison between a GPT-repaired flaky test (right) and its original developer repair (left), demonstrating a case where the GPT-repaired test passed upon execution despite displaying a very low CodeBLEU score of 36%.

```
@Test public void testFetchToAttributesWithStringValues(){
  final Map<String,String> cells=new HashMap<>();
  cells.put("cq1","val1");
  cells.put("cq2","val2");
  final long ts1=123456789;
  hBaseClientService.addResult("row1",cells,ts1);
  runner.setProperty(FetchHBaseRow.AUTHORIZATIONS,"");
  runner.setProperty(FetchHBaseRow.TABLE_NAME,"table1");
  runner.setProperty(FetchHBaseRow.ROW_ID,"row1");
  runner.setProperty(FetchHBaseRow.DESTINATION,FetchHBaseRow.DESTINATION_ATTRIBUTES);
  runner.enqueue("trigger flow file");
  runner.run();
  runner.assertTransferCount(FetchHBaseRow.REL_FAILURE,0);
  runner.assertTransferCount(FetchHBaseRow.REL_SUCCESS,1);
  runner.assertTransferCount(FetchHBaseRow.REL_NOT_FOUND,0);
  final MockFlowFile flowFile=runner.getFlowFilesForRelationship(FetchHBaseRow.REL_SUCCESS).get(0);
  flowFile.assertAttributeEquals(FetchHBaseRow.HBASE_ROW_ATTR,"{\"row\":\"row1\", \"cells\": [" + "{\"fam\":\"nifi\",\"qual\":\"cq1\",\"val\":\"val1\",\"ts\":" + ts1 + "}, "+ "{\"fam\":\"nifi\",\"qual\":\"cq2\",\"val\":\"val2\",\"ts\":"+ ts1+ "}]}");
  Assert.assertEquals(1,hBaseClientService.getNumScans());
 }
```

Fig. 13. Flaky test example with correct mapping to Change Data Structure fix category as HashMap here should be replaced with LinkedHashMap to remove flakiness.

2) GPT-Generated Repair of Flaky Tests: To examine repaired flaky tests generated by GPT, we analyze three prompting methods: (a) a simple prompt without the fix category label, (b) the same prompt but with the label, and (c) adding extra context (sample fixes) to the prompt, i.e., in-context learning (described in Section III-C3).

Fig. 17 depicts a flaky test and its fix by the original developer. Both UniXcoder and CodeBERT classified this as a *Change Data Structure* fix, replacing HashMap with LinkedHashMap to maintain the elements' order. Fig. 18 shows the results based on prompting methods (a) without the *Change Data Structure* label and (b) with that label.

When the model was provided with the fix category label, the output matched the developer's repair (Fig. 17 – right), achieving a 100% CodeBLEU Score. However, without the label indicating which statements to fix, the model suggested fixes on the wrong statements unrelated to the issue.

Now let's look at another example from the *Change Condition* category, showcased in Fig. 19. Here, the repair modified *.containsExactly()* to *.containsExactlyInAnyOrder()* to fix flakiness. Fig. 20 shows the repaired tests with and without the fix category label. It also exemplifies a repair where we provided guidance within the prompt for fixing flaky tests caused by change conditions.

In all generated instances, the generated fix is not an exact match to the original developer's repair. In the first two cases,

```
@Test public void testAllowNullSchema(){
  JSONSchema<Foo> jsonSchema=JSONSchema.of(SchemaDefinition.<Foo>builder().withPojo(Foo.class).build());
  Assert.assertEquals(jsonSchema.getSchemaInfo().getType(),SchemaType.JSON);
  Schema.Parser parser=new Schema.Parser();

  String schemaJson=new String(jsonSchema.getSchemaInfo().getSchema());
  Assert.assertEquals(schemaJson,SCHEMA_JSON_ALLOW_NULL);
  Schema schema=parser.parse(schemaJson);
  for (  String fieldName : FOO_FIELDS) {
    Schema.Field field=schema.getField(fieldName);
    Assert.assertNotNull(field);
    if (field.name().equals("field4")) {
      Assert.assertNotNull(field.schema().getTypes().get(1).getField("field1"));
    }
    if (field.name().equals("fieldUnableNull")) {
      Assert.assertNotNull(field.schema().getType());
    }
  }
}
```

(a) Flaky Test

```
@Test public void testAllowNullSchema() throws JSONException {
  JSONSchema<Foo> jsonSchema=JSONSchema.of(SchemaDefinition.<Foo>builder().withPojo(Foo.class).build());
  Assert.assertEquals(jsonSchema.getSchemaInfo().getType(),SchemaType.JSON);
  Schema.Parser parser=new Schema.Parser();
  parser.setValidateDefaults(false);
  String schemaJson=new String(jsonSchema.getSchemaInfo().getSchema());
  assertJSONEqual(schemaJson,SCHEMA_JSON_ALLOW_NULL);
  Schema schema=parser.parse(schemaJson);
  for (  String fieldName : FOO_FIELDS) {
    Schema.Field field=schema.getField(fieldName);
    Assert.assertNotNull(field);
    if (field.name().equals("field4")) {
      Assert.assertNotNull(field.schema().getTypes().get(1).getField("field1"));
    }
    if (field.name().equals("fieldUnableNull")) {
      Assert.assertNotNull(field.schema().getType());
    }
  }
}
```

(b) Fixed Flaky Test

Fig. 14. Example of Flaky test (Top) and its developer repair (Bottom) where flakiness is removed by changing assertion and adding exception handling. Our prediction model correctly classifies this flaky test under the Handle Exception and Change Assertion fix category.

.containsExactly() was replaced with *.containsAnyof()*, which is similar to the ground truth, but does not match exactly. The label-free model wrongly removed the exception handling.

```
@Test public void manyParameters() throws Exception {
  assertThat(testResult(ManyParameters.class),isSuccessful());
  assertEquals(16,ManyParameters.iterations);
  assertEquals(asList(-1,-2,-4),ManyParameters.firstTestCases.subList(0,3));
  assertEquals(asList(-1,-2,-4),ManyParameters.firstTestCases.subList(4,7));
  assertEquals(asList(-1,-2,-4),ManyParameters.firstTestCases.subList(8,11));
  assertEquals(asList(-1,-2,-4),ManyParameters.firstTestCases.subList(12,15));
  assertEquals(asList('r','r','r','r','y','y','y','y'),ManyParameters.secondTestCases.subList(0,8));
}
```

(a) Flaky test

```
@Test public void manyParameters(){
  assertThat(testResult(ManyParameters.class),isSuccessful());
  assertEquals(16,ManyParameters.iterations);
  assertEquals(asList(-4,-2,-1),ManyParameters.firstTestCases.subList(0,3));
  assertEquals(asList(-4,-2,-1),ManyParameters.firstTestCases.subList(4,7));
  assertEquals(asList(-4,-2,-1),ManyParameters.firstTestCases.subList(8,11));
  assertEquals(asList(-4,-2,-1),ManyParameters.firstTestCases.subList(12,15));
  assertEquals(asList('r','r','r','r','y','y','y','y'),ManyParameters.secondTestCases.subList(0,8));
}
```

(b) Fixed flaky test

Fig. 15. Example of Flaky test (Top) and its developer repair (Bottom) where our prediction model is unable to correctly classify this flaky test under the Handle Exception fix category.

However, with the label, the model preserved the structure and attempted to fix the problematic statements.

In the third case, in-context learning, the model correctly replaced *.containsExactly()* with *.containsExactlyInAnyOrder()* based on what model learned from the examples we passed in the prompt. However, it also incorrectly removed proper exception handling, similar to the first instance. Based on our observation, it seems that GPT's accuracy in generating fixes depends on whether the prompt contains the fix category and the number of examples. Future work could improve this accuracy by augmenting the training data and enhancing prompt quality.

We have included all the generated results in our replication package.

B. Application of FlakyFix in Practice?

In practice, our framework can help fix tests both automatically or with some human intervention. When a test is flagged as problematic (either by a person or other tools), there are two options to fix flakiness: a manual fix by the tester, or running an automated program repair tool. In either case, our approach is helpful. If the repair is manual, the predicted categories are given to the testers as suggestions so that they can more quickly fix the bug in the test code. Alternatively, if a tool such as GPT is used to fix the test, the category labels are helpful for prompting, as shown in the previous section. However, as discussed in Section III-D3, not all GPT-repaired tests are completely correct and will pass upon execution. This is primarily reflected in the CodeBLEU score and the execution results obtained after running a subset of generated tests. We estimate, however, that a large majority of GPT-repaired test cases should pass, their percentage depending on the specific prompt being used. A failed generated test will require some manual repair for it to pass. Our analysis using edit distance reveals that, on average, 16% of tokens need to be changed in a failing test for it to exactly match the passing, original-developer repaired test. Thus, completely fixing flaky test cases does entail some degree of human effort for a minority of repaired test cases and is not entirely free. However, with newer GPT versions and better prompting, we can hope for increasingly reliable fixes and passing test cases in the future.

V. Threats to Validity

A. Construct Threats

In our study, the main potential construct validity threat is the possibility of defining incorrect or incomplete labels in RQ1, which can undermine results in RQ2 and RQ3. To reduce this potential threat, we first made the process of labeling systematic and automated with heuristics that are publicly available in the replication package for further evaluation and improvement. In addition, we also manually checked random samples of the labeled tests, to confirm the correctness of the process. Furthermore, the accuracy of the results that we obtained for our predicted fix category labels are very high for most of the fix categories, as shown in Table III. Given this, we ensured that correct predicted labels are used in different prompts to LLM in RQ3.

Additionally, For RQ2, We did not conduct a per-project analysis in our study due to the limited number of positive instances for all categories of fixes and the large number of projects in our dataset (96). This implies that for each flaky test in the test set, very few flaky tests from the same project are used in the training set, especially for a specific category. We reported overall results across all projects.

Regarding the evaluation metrics in RQ3, we used the CodeBLEU score, a common metric in text generation tasks. However, we did not execute these automated generated tests to verify their functionality. Due to dependencies in open-source projects, executing all these tests is time and computation-intensive. As a result, we have not confirmed whether the tests with higher CodeBLEU scores will pass or not, or whether they are even executable. However, improvements in CodeBLEU are unlikely to yield worse test cases, in terms of functionality or execution. Nevertheless, the inadequacy of the current metrics is a known challenge [84] in this field and future research is needed to reduce such threats.

B. Internal Threats

Internal threats to validity are about to make sure the cause-effect relation identified in the study is really there and there is no other explanation (confounding factors). In our study, the claim is that *our predicted fix categories will help fix flakiness*. The main confounding factor is about the way we prompt the GPT-3.5 model, when asking to fix a flaky test. Better results might be obtained in the future with additional guidance and more examples in the prompt. Similarly, using GPT-4 would probably lead to even better results but, as discussed in Section II-C, this would raise, on our data set, possible data leakage issues. Additionally, it would be insightful to analyze how often GPT generates a correct fix when provided with an incorrectly predicted fix category. However, as explained in Section III-B5, we were limited to executing only 35 tests. Within our sample of 35 executable test cases, we encountered only one instance where the fix category was incorrectly predicted, albeit partially correct as one of the two categories was accurately predicted. Therefore, conducting this analysis reliably is not possible with our current data.

```
@Test public void testGenerateSearchWeightString(){
  Map<String,Integer> fields=new HashMap<>();
  fields.put("one",10);
  fields.put("two",1);
  fields.put("three",15);
  String generatedString=AtlasJanusGraphIndexClient.generateSearchWeightString(fields);
  Assert.assertEquals(generatedString," one^10 two^1 three^15");
}
```

Fig. 16. Example of a flaky test where the model incorrectly classifies it in the Change Assertion fix category.

```
@Test
    public void testParamsMapToString() {
        Map<String, String[]> params = new HashMap<>();
        params.put("a", new String[]{"1", "2", "3"});
        params.put("b", new String[]{"8", "7", "6"});
        String result = UrlUtils.paramsMapToString(params);
        Assert.assertEquals(result, "&a=1,2,3&b=8,7,6");
    }
```

```
@Test
    public void testParamsMapToString() {
        Map<String, String[]> params = new LinkedHashMap<>();
        params.put("a", new String[]{"1", "2", "3"});
        params.put("b", new String[]{"8", "7", "6"});
        String result = UrlUtils.paramsMapToString(params);
        Assert.assertEquals(result, "&a=1,2,3&b=8,7,6");
    }
```

Fig. 17. Example of a flaky test (left) and its original developer repair (right). Here flakiness is removed by changing the data structure i.e. replacing HashMap with LinkedHashMap.

```
@Test
public void testParamsMapToString() {
    Map<String, String[]> params = new HashMap<>();
    params.put("a", new String[]{"1", "2", "3"});
    params.put("b", new String[]{"8", "7", "6"});
    String result = UrlUtils.paramsMapToString(params);
    String expectedResult = "&a=1,2,3&b=8,7,6";
    Assert.assertTrue(result.contains("&a=1,2,3"));
    Assert.assertTrue(result.contains("&b=8,7,6"));
}
```

```
@Test
public void testParamsMapToString() {
    Map<String, String[]> params = new LinkedHashMap<>();
    params.put("a", new String[]{"1", "2", "3"});
    params.put("b", new String[]{"8", "7", "6"});
    String result = UrlUtils.paramsMapToString(params);
    Assert.assertEquals(result, "&a=1,2,3&b=8,7,6");
}

```

Fig. 18. Comparison between GPT-generated repaired flaky tests without a fix category label prompting (left) and with a fix category label provided (right), illustrating GPT's accurate generation of the repaired flaky test when given the fix category label in the prompt.

```
@Test
    public void testBuildTagSome() {
        try {
            SubscriptionData subscriptionData = FilterAPI.build(
                    "TOPIC", "A || B", ExpressionType.TAG
            );
            assertThat(subscriptionData).isNotNull();
            assertThat(subscriptionData.getTopic()).isEqualTo("TOPIC");
            assertThat(subscriptionData.getSubString()).isEqualTo("A || B");
            assertThat(ExpressionType.isTagType(subscriptionData.getExpressionType())).isTrue();

assertThat(subscriptionData.getTagsSet()).isNotNull();
            assertThat(subscriptionData.getTagsSet()).containsExactly("A", "B");
        } catch (Exception e) {
            e.printStackTrace();
            assertThat(Boolean.FALSE).isTrue();
        }
    }
```

```
@Test
    public void testBuildTagSome() {
        try {
            SubscriptionData subscriptionData = FilterAPI.build(
                    "TOPIC", "A || B", ExpressionType.TAG
            );
            assertThat(subscriptionData).isNotNull();
            assertThat(subscriptionData.getTopic()).isEqualTo("TOPIC");
            assertThat(subscriptionData.getSubString()).isEqualTo("A || B");
            assertThat(ExpressionType.isTagType(subscriptionData.getExpressionType())).isTrue();

assertThat(subscriptionData.getTagsSet()).isNotNull();
            assertThat(subscriptionData.getTagsSet()).containsExactlyInAnyOrder("A", "B");
        } catch (Exception e) {
            e.printStackTrace();
            assertThat(Boolean.FALSE).isTrue();
        }
    }
```

Fig. 19. Example of a flaky test (left) and its original developer repair (right). Here flakiness is removed by changing the condition, i.e., replacing .containsExactly() with .containsExactlyInAnyOrder().

```java
@Test
public void testBuildTagSome() {
    try {
        SubscriptionData subscriptionData = FilterAPI.build(
                "TOPIC", "A || B", ExpressionType.TAG
        );
        assertThat(subscriptionData).isNotNull();
        assertThat(subscriptionData.getTopic()).isEqualTo("TOPIC");
        assertThat(subscriptionData.getSubString()).isEqualTo("A || B");
        assertThat(ExpressionType.isTagType(subscriptionData.getExpressionType
())).isTrue();

        assertThat(subscriptionData.getTagsSet()).isNotNull();
        assertThat(subscriptionData.getTagsSet()).containsAnyOf("A", "B");
    } catch (Exception e) {
        e.printStackTrace();
        assertThat(false).isTrue();
    }
}
```

(a) Repaired flaky test with fix category label.

```java
@Test
public void testBuildTagSome() {
    SubscriptionData subscriptionData = FilterAPI.build(
            "TOPIC", "A || B", ExpressionType.TAG
    );
    assertThat(subscriptionData).isNotNull();
    assertThat(subscriptionData.getTopic()).isEqualTo("TOPIC");
    assertThat(subscriptionData.getSubString()).isEqualTo("A || B");
    assertThat(ExpressionType.isTagType(subscriptionData.getExpressionType())).
isTrue();
    assertThat(subscriptionData.getTagsSet()).isNotNull();
    assertThat(subscriptionData.getTagsSet()).containsAnyOf("A", "B");
}
```

(b) Repaired flaky test without fix category label.

```java
@Test
public void testBuildTagSome() {
    SubscriptionData subscriptionData = FilterAPI.build(
            "TOPIC", "A || B", ExpressionType.TAG
    );
    assertThat(subscriptionData).isNotNull();
    assertThat(subscriptionData.getTopic()).isEqualTo("TOPIC");
    assertThat(subscriptionData.getSubString()).isEqualTo("A || B");
    assertThat(ExpressionType.isTagType(subscriptionData.getExpressionType())).
isTrue();

    assertThat(subscriptionData.getTagsSet()).isNotNull();
    assertThat(subscriptionData.getTagsSet()).containsExactlyInAnyOrder("A",
"B");
}
```

(c) Repaired flaky test with in-context learning.

Fig. 20. Effect of different prompts on GPT's repaired flaky test generation: with fix label (Top), without fix label (middle), and with in-context learning (bottom).

Another internal validity threat while predicting the labels is the models context size (510 tokens for CodeBERT and 1022 for UniXcoder), which may truncate the source code of some test cases and result in information loss. In our dataset, 49 tests (11% of the total 562 flaky tests) exceed the token size limit of 510 tokens, and 17 tests (4%) exceed 1022 tokens. This could potentially impact the accuracy of our prediction results, especially for those fix categories where we have limited positive examples. Further research should investigate the extent to which this limitation may affects the outcomes of our study.

C. Conclusion Threats

To reduce potential conclusion validity threats, in RQ2, we used Fisher's Exact test to make sure our conclusions are statistically significant. We also ran Wilcoxon signed-rank test when comparing different prompt types in RQ3. However, there is still a potential conclusion threat in RQ3 results with respect to the inherent randomness of GPT models. Indeed, using the same model and the same prompt can result in different outputs when using higher temperatures. In general, the field of prompt engineering, which also includes the optimization of outputs with several rounds of prompting, is a new field and there are limited precise and reliable guidelines in the literature. Future work will investigate future prompting strategies as they get reported.

D. External Threats

Our automated labeling tool and the heuristics are defined for Java tests. We have not evaluated our tool on other programming languages but it can be adapted to other languages and new projects by incorporating adapted keywords and rules. Additionally, CodeBERT, UniXcoder and GPT models are pre-trained on multiple programming languages.

VI. Related Work

In this section, we provide an overview of the most related work to predicting flaky test fix categories. There are two areas of research that we deliberately exclude, since they are less related to our study. The first category that we do not include is generic program repair studies [85]. Although, fixing test code can be seen as a similar task to program repair, there are fundamental differences between the two areas which are mainly due to the patterns of fixes. A generic code repair can come in many flavors and patterns and thus predicting a category of fix is a daunting task. In addition, there are far more data available for generic program repair (e.g., literally any code fix in GitHub) than for fixing test flakiness. Therefore, we do not discuss individual papers that study generic program repair. The second category that we exclude here are studies that analyze test smells and their patterns [86]. Although these works are more relevant since they are about issues in the test code, given that they are again generic and not related to flakiness, we exclude them.

The first set of relevant papers to our study are those that specifically address categories or taxonomies of fixes or fix patterns. Although they are not about flaky test fixes or not even about test cases, these studies have nevertheless a similar problem to solve: categorizing code fixes or causes and predicting them. Ni et al. [71] proposed an approach to classify bug fix categories using a Tree-based Convolutional Neural Network (TBCNN). To do so, they used the *diff* of the source code before and after the fix at the Abstract Syntax Tree (AST) level to construct fix trees as input for TBCNN. In contrast, our

objective is to develop a classifier for fix categories that only use flaky test code to make predictions, without relying on the repaired versions or production code.

Predicting the cause of flakiness is another category that is relevant to our work. Akli et al. proposed a novel approach [23] for classifying flaky tests based on their root cause category using CodeBERT and FSL. They manually labeled 343 flaky tests with 10 different categories of flaky test causes suggested by Luo et al. [1] and Eck et al. [63]. However, they reported the prediction results for only four categories of causes, namely Async wait, Concurrency, Time, and Unordered collection, as they lacked sufficient positive examples to train an ML-based classifier for the other categories. They achieved the highest prediction results for the Unordered collection cause, with a precision of 85% and a recall of 80%. In a recent survey on flaky tests [19], [87], Parry et al. mapped three different causes of flakiness with their potential repairs. For instance, they suggested adding or modifying assertions in the test code to fix the flakiness caused by concurrency. Predicting fix categories provides testers with a specific direction for modifying the code. Knowing that concurrency is causing flakiness in a given test is not as useful to testers as they do not know which part of the code is causing the issue. However, knowing that changing assertions can eliminate flakiness gives them more practical guidance. Other researchers reported studies [39], [87] to identify flakiness causes, impacts, and existing strategies to fix flakiness. However, none of them attempted to categorize the tests based on their fixes. Research on repairing flakiness and categorizing fix categories is still in its early stages, with much of the work focused on fixing order-dependent flaky tests. For example, iPFlakies [64] and iFixFlakies [35] propose methods for addressing order-dependent flakiness, while Flex [62] automates the repair of flaky tests caused by algorithmic randomness in ML algorithms. Parry et al. [20] proposed a study in which different categories of flaky test fixes were defined and 75 flaky commits were categorized based on these categories. However, the categorization was performed through manual inspection of the commits, code diffs, developer comments, and other linked issues outside the scope of the test code. In another study of flaky tests in the Mozilla database [63] tests were categorized based on their origin (i.e., the root cause of the flakiness, whether in the source or test code) and developers' efforts to fix them. The need for automated fixing of flaky tests and conducting quick small repairs was strongly suggested in a recent survey on supporting developers and testers in addressing test flakiness [88]. Our study aims to address these gaps by developing an automated solution to categorize flaky tests based on well-defined fix categories, relying exclusively on test code, providing practical guidance for both manual and automated repair. We also provide a way to automatically label flaky tests with selected, common fix categories to train context-specific prediction models.

Although we do not review the generic automated program repair literature, as explained, we do however discuss related work where LLMs are used for automatic program repair, since it is very relevant to RQ3 from a technical standpoint. Recent advancements in LLMs, exemplified by ChatGPT [57], Copilot [89], and similar models tailored for code, have significantly advanced this field. These models exhibit adeptness in code generation and are generating considerable interest among researchers striving to enhance automatic program repair methodologies.

Ribeiro et al. [90] employed GPT-3 to automatically rectify bugs in 1318 buggy OCaml programs, achieving a repair rate of 39%. Julian et al. [91], using OpenAI's Codex model, repaired bugs in Java and Python programs, noting a higher repair rate for Python. Their utilization of prompts and hints improved Codex's bug identification, yielding slightly enhanced results in repairing Java programs. In another recent study by Chunqiu et al. [92], different LLMs like Codex, InCoder, CodeT5 and GPT-2 are investigated for program repair in Java, Python, and C programs.

Additionally, Mashadi et al. [76] and Xia et al. [93] utilized CodeBERT for automated program repair on datasets like ManySStuBs4J and Defects4J, respectively, achieving state-of-the-art results in bug repair for smaller code segments due to CodeBERT's input limitation of up to 512 tokens.

However, while existing research has focused on rectifying bugs in source code, little attention has been directed towards addressing flakiness within test code itself. Our study marks the first attempt to fully automate flaky test repairs using GPT, complemented by providing fix category information to pinpoint flakiness related statements in the code and enhance the repair capability of the GPT model.

VII. Conclusion and Future Work

In this paper, we proposed a framework to (a) categorize flaky tests according to practical fix categories based exclusively on test code and (b) use the fix categories to generate fully automated repairs of the flaky tests with language models. Given the limited availability of open-source datasets for flaky test research, our generated labeled data can also be valuable for future research, particularly for ML-based prediction and repair of flaky tests. We evaluated two language models, CodeBERT and UniXcoder, with and without Few-Shot Learning, and found that the UniXcoder model outperformed CodeBERT in predicting most fix categories. We also showed that the category information, complemented with in-context learning can greatly help a large language model (LLM), namely GPT-3.5 in our case, better fix the flakiness in test code. Lastly, we executed a sample of GPT-repaired flaky tests and estimated that a large percentage (roughly between 51% and 83%) of these tests can be expected to pass. For the generated tests that fail, an average of 16% of the code needs further modification for them to pass. In the future, we plan to expand our flaky dataset with more data. We also plan to expand our heuristics for automatically labeling flaky tests to other programming languages. Replicating the study with other LLMs (such as GPT-4.0) will also be made possible with an updated dataset. Additionally, a significant percentage of flaky tests may stem from problems in the production code, which cannot be addressed by black-box models. Therefore, in the future, we need to develop lightweight

and scalable approaches for accessing and fixing the production code when dealing with test flakiness.

Data Availability Statement

The experiments conducted in this work were enabled in part by WestGrid (https://www.westgrid.ca) and Compute Canada (https://www.computecanada.ca).

References

[1] Q. Luo, F. Hariri, L. Eloussi, and D. Marinov, "An empirical analysis of flaky tests," in *Proc. 22nd ACM SIGSOFT Int. Symp. Found. Softw. Eng.*, 2014, pp. 643–653.

[2] E. Fallahzadeh and P. C. Rigby, "The impact of flaky tests on historical test prioritization on chrome," in *Proc. 44th Int. Conf. Softw. Eng.: Softw. Eng. Pract.*, 2022, pp. 273–282.

[3] B. Harry, "How we approach testing VSTS to enable continuous delivery," 2019. [Online]. Available: https://devblogs.microsoft.com/bharry/testing-in-a-cloud-delivery-cadence/

[4] M. Cordy, R. Rwemalika, M. Papadakis, and M. Harman, "FlakiMe: Laboratory-controlled test flakiness impact assessment. A case study on mutation testing and program repair," 2019, *arXiv:1912.03197*.

[5] W. Lam, S. Winter, A. Wei, T. Xie, D. Marinov, and J. Bell, "A large-scale longitudinal study of flaky tests," *Proc. ACM Program. Lang.*, vol. 4, no. OOPSLA, pp. 1–29, 2020.

[6] S. Habchi, G. Haben, M. Papadakis, M. Cordy, and Y. L. Traon, "A qualitative study on the sources, impacts, and mitigation strategies of flaky tests," in *Proc. IEEE Conf. Softw. Testing, Verification Validation (ICST)*, Piscataway, NJ, USA: IEEE Press, 2022, pp. 244–255.

[7] A. Ahmad, F. G. Oliveira Neto, Z. Shi, K. Sandahl, and O. Leifler, "A multi-factor approach for flaky test detection and automated root cause analysis," in *Proc. 28th Asia-Pacific Softw. Eng. Conf. (APSEC)*, Piscataway, NJ, USA: IEEE Press, 2021, pp. 338–348.

[8] R. Verdecchia, E. Cruciani, B. Miranda, and A. Bertolino, "Know you neighbor: Fast static prediction of test flakiness," *IEEE Access*, vol. 9, pp. 76119–76134, 2021.

[9] V. Pontillo, F. Palomba, and F. Ferrucci, "Toward static test flakiness prediction: A feasibility study," in *Proc. 5th Int. Workshop Mach. Learn. Techn. Softw. Qual. Evol.*, 2021, pp. 19–24.

[10] J. Bell, O. Legunsen, M. Hilton, L. Eloussi, T. Yung, and D. Marinov, "DeFlaker: Automatically detecting flaky tests," in *Proc. 40th Int. Conf. Softw. Eng.*, 2018, pp. 433–444.

[11] Y. Qin et al., "Peeler: Learning to effectively predict flakiness without running tests," in *Proc. IEEE Int. Conf. Softw. Maintenance Evol. (ICSME)*, Piscataway, NJ, USA: IEEE Press, 2022, pp. 257–268.

[12] S. Fatima, T. A. Ghaleb, and L. Briand, "Flakify: A black-box, language model-based predictor for flaky tests," *IEEE Trans. Softw. Eng.*, vol. 49, no. 4, pp. 1912–1927, Apr. 2023.

[13] M. Gruber, M. Heine, N. Oster, M. Philippsen, and G. Fraser, "Practical flaky test prediction using common code evolution and test history data," in *Proc. IEEE Conf. Softw. Testing, Verification Validation (ICST)*, Piscataway, NJ, USA: IEEE Press, 2023, pp. 210–221.

[14] A. Alshammari, C. Morris, M. Hilton, and J. Bell, "FlakeFlagger: Predicting flakiness without rerunning tests," in *Proc. IEEE/ACM 43rd Int. Conf. Softw. Eng. (ICSE)*, Piscataway, NJ, USA: IEEE Press, 2021, pp. 1572–1584.

[15] G. Pinto, B. Miranda, S. Dissanayake, M. d'Amorim, C. Treude, and A. Bertolino, "What is the vocabulary of flaky tests?" in *Proc. 17th Int. Conf. Mining Softw. Repositories*, 2020, pp. 492–502.

[16] G. Haben, S. Habchi, M. Papadakis, M. Cordy, and Y. L. Traon, "A replication study on the usability of code vocabulary in predicting flaky tests," in *Proc. IEEE/ACM 18th Int. Conf. Mining Softw. Repositories (MSR)*, Piscataway, NJ, USA: IEEE Press, 2021, pp. 219–229.

[17] B. H. P. Camara, M. Auré, G. Silva, A. T. Endo, and S. R. Vergilio, "What is the vocabulary of flaky tests? An extended replication," in *Proc. IEEE/ACM 29th Int. Conf. Program Comprehension (ICPC)*, Piscataway, NJ, USA: IEEE Press, 2021, pp. 444–454.

[18] A. Ahmad, "An evaluation of machine learning methods for predicting flaky tests," in *Proc. 27th Asia-Pacific Softw. Eng. Conf. (APSEC)*, Singapore, Dec. 2020, pp. 3–9.

[19] O. Parry, G. M. Kapfhammer, M. Hilton, and P. McMinn, "A survey of flaky tests," *ACM Trans. Softw. Eng. Methodol.*, vol. 31, no. 1, pp. 1–74, 2021.

[20] O. Parry, G. M. Kapfhammer, M. Hilton, and P. McMinn, "What do developer-repaired flaky tests tell us about the effectiveness of automated flaky test detection?" in *Proc. 3rd ACM/IEEE Int. Conf. Automat. Softw. Test*, 2022, pp. 160–164.

[21] N. Hashemi, A. Tahir, and S. Rasheed, "An empirical study of flaky tests in JavaScript," in *Proc. IEEE Int. Conf. Softw. Maintenance Evol. (ICSME)*, Piscataway, NJ, USA: IEEE Press, 2022, pp. 24–34.

[22] W. Lam, "International dataset of flaky tests (IDoFT)," 2020. [Online]. Available: https://github.com/TestingResearchIllinois/idoft

[23] A. Akli, G. Haben, S. Habchi, M. Papadakis, and Y. L. Traon, "FlakyCat: Predicting flaky tests categories using few-shot learning," in *Proc. IEEE/ACM Int. Conf. Automat. Softw. Test (AST)*, Piscataway, NJ, USA: IEEE Press, 2023, pp. 140–151.

[24] K. Wing Lam, M. vanç, H. S. lu, and S. Thummalapenta, "A study on the lifecycle of flaky tests," in *Proc. ACM/IEEE 42nd Int. Conf. Softw. Eng.*, 2020, pp. 1471–1482.

[25] Z. Feng et al., "CodeBERT: A pre-trained model for programming and natural languages," 2020, *arXiv:2002.08155*.

[26] D. Guo et al., "UniXcoder: Unified cross-modal pre-training for code representation," 2022, *arXiv:2203.03850*.

[27] Y. Wang, Q. Yao, J. T. Kwok, and L. M. Ni, "Generalizing from a few examples: A survey on few-shot learning," *ACM Comput. Surveys*, vol. 53, no. 3, pp. 1–34, 2020.

[28] R. OpenAI, "GPT-4 technical report," 2023, *arxiv:2303.08774*.

[29] Y. Dong, J. Ding, X. Jiang, Z. Li, G. Li, and Z. Jin, "CodeScore: Evaluating code generation by learning code execution," 2023, *arXiv:2301.09043*.

[30] J. Liu, D. Shen, Y. Zhang, B. Dolan, L. Carin, and W. Chen, "What makes good in-context examples for GPT-3?" 2021, *arXiv:2101.06804*.

[31] A. Wei, P. Yi, T. Xie, D. Marinov, and W. Lam, "Probabilistic and systematic coverage of consecutive test-method pairs for detecting order-dependent flaky tests," in *Proc. Tools Algorithms Construction Anal. Syst.: 27th Int. Conf., (TACAS),* Luxembourg City, Luxembourg, Springer, Mar. 2013, pp. 270–287.

[32] W. Lam, R. Oei, A. Shi, D. Marinov, and T. Xie, "iDFlakies: A framework for detecting and partially classifying flaky tests," in *Proc. 12th IEEE Conf. Softw. Testing, Validation Verification*, Piscataway, NJ, USA: IEEE Press, 2019, pp. 312–322.

[33] W. Lam, S. Winter, A. Astorga, V. Stodden, and D. Marinov, "Understanding reproducibility and characteristics of flaky tests through test reruns in Java projects," in *Proc. IEEE 31st Int. Symp. Softw. Rel. Eng. (ISSRE)*, Piscataway, NJ, USA: IEEE Press, 2020, pp. 403–413.

[34] W. Lam, A. Shi, R. Oei, S. Zhang, M. D. Ernst, and T. Xie, "Dependent-test-aware regression testing techniques," in *Proc. 29th ACM SIGSOFT Int. Symp. Softw. Testing Anal.*, 2020, pp. 298–311.

[35] A. Shi, W. Lam, R. Oei, T. Xie, and D. Marinov, "iFixFlakies: A framework for automatically fixing order-dependent flaky tests," in *Proc. 27th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2019, pp. 545–555.

[36] M. L. McHugh, "Interrater reliability: The kappa statistic," *Biochemia Medica*, vol. 22, no. 3, pp. 276–282, 2012.

[37] G. Meszaros, *xUnit Test Patterns: Refactoring Test Code*. London, U.K.: Pearson Education, 2007.

[38] R. Osherove, *The Art of Unit Testing: With Examples in C. Simon and Schuster*, Shelter Island, New York, USA: Manning, 2013.

[39] W. Zheng, G. Liu, M. Zhang, X. Chen, and W. Zhao, "Research progress of flaky tests," in *Proc. IEEE Int. Conf. Softw. Anal., Evol. Reeng. (SANER)*, Piscataway, NJ, USA: IEEE Press, 2021, pp. 639–646.

[40] Y. Pei, J. Sohn, S. Habchi, and M. Papadakis, "TRaf: Time-based repair for asynchronous wait flaky tests in web testing," 2023, *arXiv:2305.08592*.

[41] C. Niu, C. Li, V. Ng, D. Chen, J. Ge, and B. Luo, "An empirical comparison of pre-trained models of source code," 2023, *arXiv:2302.04026*.

[42] W. X. Zhao et al., "A survey of large language models," 2023, *arXiv:2303.18223*.

[43] R. Sennrich, B. Haddow, and A. Birch, "Neural machine translation of rare words with subword units," 2015, *arXiv:1508.07909*.

[44] A. He, C. Luo, X. Tian, and W. Zeng, "A twofold Siamese network for real-time object tracking," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 4834–4843.

[45] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A unified embedding for face recognition and clustering," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2015, pp. 815–823.

[46] P. Neculoiu, M. Versteegh, and M. Rotaru, "Learning text similarity with Siamese recurrent networks," in *Proc. 1st Workshop Representation Learn. NLP*, 2016, pp. 148–157.

[47] G. Koch, R. Zemel, and R. Salakhutdinov, "Siamese neural networks for one-shot image recognition," in *Proc. ICML Deep Learn. Workshop*, vol. 2, 2015, pp. 3–7.

[48] J. Mueller and A. Thyagarajan, "Siamese recurrent architectures for learning sentence similarity," in *Proc. AAAI Conf. Artif. Intell.*, vol. 30, 2016, 2–6.

[49] S. E. Anigri, M. M. Himmi, and A. Mahmoudi, "How BERT's dropout fine-tuning affects text classification?" in *Proc. Int. Conf. Bus. Intell.*, Springer, 2021, pp. 130–139.

[50] Z. Yao, A. Gholami, S. Shen, M. Mustafa, K. Keutzer, and M. Mahoney, "ADAHESSIAN: An adaptive second order optimizer for machine learning," in *Proc. AAAI Conf. Artif. Intell.*, vol. 35, 2021, pp. 10665–10673.

[51] A. F. Agarap, "Deep learning using rectified linear units," 2018, *arXiv:1803.08375*.

[52] Z. Zheng et al., "A survey of large language models for code: Evolution, benchmarking, and future trends," 2023, *arXiv:2311.10372*.

[53] A. Fan et al., "Large language models for software engineering: Survey and open problems," 2023, *arXiv:2310.03533*.

[54] H. Pearce, B. Tan, B. Ahmad, R. Karri, and B. Dolan-Gavitt, "Examining zero-shot vulnerability repair with large language models," in *Proc. IEEE Symp. Secur. Privacy (SP)*, Piscataway, NJ, USA: IEEE Press, 2023, pp. 2339– 2356.

[55] J. White, S. Hays, Q. Fu, J. Spencer-Smith, and D. C. Schmidt, "Chatgpt prompt patterns for improving code quality, refactoring, requirements elicitation, and software design," 2023, *arXiv:2303.07839*.

[56] Q. Zhang et al., "A critical review of large language model on software engineering," 2023, *arXiv:2310.08879*.

[57] M. Fu, C. Tantithamthavorn, Nguyen, and T. Le, "ChatGPT for vulnerability detection, classification, and repair," 2023, *arXiv:2310.09810*.

[58] "OpenAI platform documentation: Continuous model upgrades," OpenAI. Accessed: 2023. [Online]. Available: https://platform.openai.com/docs/overview

[59] S. Min et al., "Rethinking the Role of Demonstrations: What Makes In-Context Learning Work?" in *Proc. Conf. Empirical Methods Nat. Lang. Process.*, 2022, *arXiv:2202.12837*.

[60] S. Qiao et al., "Reasoning with language model prompting: A survey," 2022, *arXiv:2212.09597*.

[61] T. Brown et al., "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 1877–1901.

[62] S. Dutta, A. Shi, and S. Misailovic, "FLEX: Fixing flaky tests in machine learning projects by updating assertion bounds," in *Proc. 29th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2021, pp. 603–614.

[63] M. Eck, F. Palomba, M. Castelluccio, and A. Bacchelli, "Understanding flaky tests: The developer's perspective," in *Proc. 27th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, pp. 830–840, 2019.

[64] R. Wang, Y. Chen, and W. Lam, "iPFlakies: A framework for detecting and fixing python order-dependent flaky tests," in *Proc. ACM/IEEE 44th Int. Conf. Softw. Eng.: Companion Proc.*, 2022, pp. 120–124.

[65] P. Branco, s. T. Luí, and R. P. Ribeiro, "A survey of predictive modeling on imbalanced domains," *ACM Comput. Surveys (CSUR)*, vol. 49, no. 2, pp. 1–50, 2016.

[66] C. Shorten, T. M. Khoshgoftaar, and B. Furht, "Text data augmentation for deep learning," *J. Big Data*, vol. 8, no. 101, pp. 1–34, 2021.

[67] T. Verdonck, B. Baesens, Ma. óskarsdóttir, and S. vanden Broucke, "Special issue on feature engineering editorial," *Mach. Learn.*, vol. 113, no. 7, pp. 3917–3928, 2021.

[68] R. Pascanu, T. Mikolov, and Y. Bengio, "On the difficulty of training recurrent neural networks," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2013, pp. 1310–1318.

[69] R. Collobert et al., "Natural language processing (almost) from scratch," *J. Mach. Learn. Res.*, vol. 12, no. ARTICLE, pp. 2493–2537, 2011.

[70] J. Micco, "Flaky tests at Google and how we mitigate them," Googblogs, 2016 [Online]. Available: https://testing.googleblog.com/2016/05/flaky-tests-at-google-and-how-we.html.

[71] F. Thung, D. Lo, and L. Jiang, "Automatic defect categorization," in *Proc. 19th Work. Conf. Reverse Eng.,* Piscataway, NJ, USA: IEEE Press, 2012, pp. 205–214.

[72] M. Raymond and R. Franç, "An exact test for population differentiation," *Evolution*, vol. 49, no. 6, pp. 1280–1283, 1995.

[73] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "BLEU: A method for automatic evaluation of machine translation," in *Proc. 40th Annu. Meeting Assoc. Comput. Linguistics*, 2002, pp. 1–7.

[74] M. M. Abdollahpour, M. Ashtiani, and F. Bakhshi, "Automatic software code repair using deep learning techniques," *Softw. Quality J.*, vol. 32, no. 2, pp. 1–30, 2023.

[75] C. Liu et al., "Improving chatGPT prompt for code generation," 2023, *arXiv:2305.08360*.

[76] E. Mashhadi and H. Hemmati, "Applying codeBERT for automated program repair of Java simple bugs," in *Proc. IEEE/ACM 18th Int. Conf. Mining Softw. Repositories (MSR)*, Piscataway, NJ, USA: IEEE Press, 2021, pp. 505–509.

[77] S. Ren et al., "CodeBLEU: A method for automatic evaluation of code synthesis," 2020, *arXiv:2009.10297*.

[78] C. Z. Mooney, R. D. Duval, and R. Duvall, *Bootstrapping: A Non-parametric Approach to Statistical Inference*. New York, USA: Sage Publications, no. 95, 1993.

[79] D. W. Hosmer, Jr, S. Lemeshow, and R. X. Sturdivant, *Applied Logistic Regression*. Hoboken, NJ, USA: Wiley, 2013.

[80] E. S. Ristad and P. N. Yianilos, "Learning string-edit distance," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 20, no. 5, pp. 522–532, May 1998.

[81] R. F. Woolson, "Wilcoxon signed-rank test," in *Wiley Encyclopedia of Clinical* Trials, Hoboken, NJ, USA: Wiley, 2007, pp. 1–3.

[82] M. L. McHugh, "The chi-square test of independence," *Biochemia Medica*, vol. 23, no. 2, pp. 143–149, 2013.

[83] S. F. Assmann, D. W. Hosmer, S. Lemeshow, and K. A. Mundt, "Confidence intervals for measures of interaction," Epidemiol., vol. 7, no. 3, pp. 286–290, 1996.

[84] M. Evtikhiev, E. Bogomolov, Y. Sokolov, and T. Bryksin, "Out of the BLEU: How should we assess quality of the code generation models?" *J. Syst. Softw.*, vol. 203, no. 111741, pp. 2–12, 2023.

[85] C. L. Goues, M. Pradel, and A. Roychoudhury, "Automated program repair," *Commun. ACM*, vol. 62, no. 12, pp. 56–65, 2019.

[86] V. Garousi and B. Küçük, "Smells in software test code: A survey of knowledge in industry and academia," *J. Syst. Softw.*, vol. 138, pp. 52–81, Apr. 2018.

[87] W. Lam, P. Godefroid, S. Nath, A. Santhiar, and S. Thummalapenta, "Root causing flaky tests in a large-scale industrial setting," in *Proc. 28th ACM SIGSOFT Int. Symp. Softw. Testing Anal.*, 2019, pp. 101–111.

[88] M. Gruber and G. Fraser, "A survey on how test flakiness affects developers and what support they need to address it," in *Proc. IEEE Conf. Softw. Testing, Verification Validation (ICST)*, Piscataway, NJ, USA: IEEE Press, 2022, pp. 82–92.

[89] N. Nguyen and S. Nadi, "An empirical evaluation of GitHub Copilot's code suggestions," in *Proc. 19th Int. Conf. Mining Softw. Repositories*, 2022, pp. 1–5.

[90] F. Ribeiro, N. C. d. M. José, K. Tsushima, R. Abreu, and o. S. Joã, "GPT-3-powered type error debugging: Investigating the use of large language models for code repair," in *Proc. 16th ACM SIGPLAN Int. Conf. Softw. Lang. Eng.*, 2023, pp. 2–11.

[91] J. A. Prenner, H. Babii, and R. Robbes, "Can openAI's codex fix bugs? An evaluation on quixbugs," in *Proc. 3rd Int. Workshop Automated Program Repair*, 2022, pp. 69–75.

[92] C. S. Xia, Y. Wei, and L. Zhang, "Automated program repair in the era of large pre-trained language models," in *Proc. 45th Int. Conf. Softw. Eng. (ICSE),* New York, NY, USA: ACM, 2023, pp. 3–9.

[93] C. S. Xia and L. Zhang, "Less training, more repairing please: Revisiting automated program repair via zero-shot learning," in *Proc. 30th ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2022, pp. 959–971.



Sakina Fatima received an Erasmus Mundus Joint master's degree in dependable software systems from the University of St. Andrews, United Kingdom and Maynooth University, Ireland. She is currently working toward the Ph.D. degree with the School of Electrical Engineering and Computer Science, University of Ottawa. She is a Member of Nanda Lab. She has worked in collaboration with Microsoft Research, IBM and Huawei. Her research interests include automated software testing, generative AI, large language model and applied machine learning. In 2019, she was awarded the French Government Medal and the National University of Ireland prize for distinction in collaborative degrees.



Hadi Hemmati received the Ph.D. degree from the University of Oslo, Norway. He is an Associate Professor with the Electrical Engineering and Computer Science Department, York University. Previously he was an Associate Professor with the Electrical and Software Engineering Department, University of Calgary, AB, Canada. In the past, he was also an Assistant Professor with the University of Manitoba, and a Postdoctoral Fellow with the University of Waterloo, and Queen's University. His main research interests are automated software engineering (with a focus on software testing, debugging, and repair), and trustworthy AI (with a focus on robustness and explainability). His research has a strong focus on pragmatic software/ML solutions for large-scale systems and empirically investigating them in practice. He has been a PI on multiple industry research projects in different domains such as IT, aviation, insurance, urban development, fintech, and beyond.



Lionel C. Briand (Fellow, IEEE) is a Professor of software engineering and has shared appointments between (1) The University of Ottawa, Canada, and (2) The Lero SFI Centre—the national Irish centre for software research—hosted by the University of Limerick, Ireland. In collaboration with colleagues, for over 30 years, he has run many collaborative research projects with companies in the automotive, satellite, aerospace, energy, financial, and legal domains. He has held various engineering, academic, and leading positions in seven countries. He currently holds a Canada Research Chair (Tier 1) on "Intelligent Software Dependability and Compliance" and is the director of Lero, the national Irish centre for software research. He was elevated to the grades of ACM Fellow for his work on software testing and verification. Further, he was granted the IEEE Computer Society Harlan Mills award, the ACM SIGSOFT outstanding research award, and the IEEE Reliability Society Engineer-of-the-year award. He also received an ERC Advanced Grant in 2016 on modelling and testing cyber-physical systems, the most prestigious individual research award in the European Union and was elected a fellow of the Academy of Science, Royal Society of Canada in 2023. For more information, see http://www.lbriand.info.